Python 编程入门与实践

（微视频版）

席　亮　毕津滔　马　俊　主编

清華大學出版社
北　京

内 容 简 介

本书以通俗易懂的语言、翔实生动的案例全面介绍使用 Python 编程的方法和技巧。本书共分 11 章，内容涵盖了 Python 编程基础，Python 语法基础，Python 控制语句，Python 函数与模块，Python 文件操作，面向对象程序设计，Python GUI 编程，Python 多线程编程技术，Python 访问数据库，Python 图像处理和 Python 网络编程等。在每章的最后都提供了大量经典的编程案例，力求为读者带来良好的学习体验。

与书中内容同步的案例操作教学视频可供读者随时扫码学习(可扫描前言中的二维码在线观看)。本书具有很强的实用性和可操作性，可以作为 Python 初学者的自学用书，也可作为 Python 开发技术人员的首选参考书，还可作为高等院校相关专业的授课教材。

本书对应的电子课件、完整代码文档和实例源文件可以到 http://www.tupwk.com.cn/downpage 网站下载，也可以通过扫描前言中的二维码获取。

本书封面贴有清华大学出版社防伪标签，无标签者不得销售。
版权所有，侵权必究。举报：010-62782989，beiqinquan@tup.tsinghua.edu.cn。

图书在版编目（CIP）数据

Python编程入门与实践 : 微视频版 / 席亮, 毕津滔, 马俊主编. -- 北京 : 清华大学出版社, 2025. 7.
ISBN 978-7-302-69558-5

Ⅰ. TP312.8

中国国家版本馆CIP数据核字第2025L9466R号

责任编辑：胡辰浩
封面设计：高娟妮
版式设计：妙思品位
责任校对：成凤进
责任印制：宋　林

出版发行：清华大学出版社
网　　址：https://www.tup.com.cn，https://www.wqxuetang.com
地　　址：北京清华大学学研大厦A座　　**邮　　编：**100084
社 总 机：010-83470000　　**邮　　购：**010-62786544
投稿与读者服务：010-62776969，c-service@tup.tsinghua.edu.cn
质 量 反 馈：010-62772015，zhiliang@tup.tsinghua.edu.cn
印 装 者：三河市铭诚印务有限公司
经　　销：全国新华书店
开　　本：185mm×260mm　　**印　张：**20.75　　**字　数：**479千字
版　　次：2025年8月第1版　　**印　次：**2025年8月第1次印刷
定　　价：89.80元

产品编号：110748-01

前　言

-preface-

Python是一种简单易学且功能强大的编程语言，具备高效的高级数据结构，能够简洁而有效地实现面向对象编程。Python的简洁语法，加之其解释型语言的特性，使其成为许多平台上理想的脚本语言，尤其适用于快速开发应用程序。

Python的应用领域广泛，包括数据分析、组件集成、网络服务、图像处理、数值计算和科学计算等。目前，众多大中型互联网企业均在使用Python，如豆瓣、知乎、百度、腾讯、美团等。这些互联网公司普遍使用Python进行自动化运维、自动化测试、大数据分析、网络爬虫和 Web 开发等。

Python 的学习和掌握相对容易，且拥有大量内置函数和丰富的扩展库，可以快速实现众多复杂功能。在学习Python语言的过程中，仍需通过不断的练习来熟悉其编程模式，建议避免将其他语言的编程风格应用于Python，而应从自然和简洁的角度出发，以免设计出冗长且低效的Python程序。

一、本书内容特点

(1) 本书专为计算机相关专业的高校学生及程序设计爱好者编写，详细介绍了Python语言的各种规则和规范，帮助读者全面掌握这门编程语言，从而设计出优秀的程序。

(2) Python程序设计涉及的范围非常广泛，本书内容的编排并不求全、求深，而是考虑零基础读者的接受能力，其中语法的介绍以够用、实用为原则，本书选择Python中实用的知识进行讲解，强化对程序思维能力的培养。

(3) 书中实例均提供了详细的设计思路、关键技术分析以及具体的解决方案。每个知识点均配有示例代码，辅以相关说明和运行结果，某些章节还深入探讨经典的程序设计问题。读者可通过参考源程序进行实践，加深理解。

需要说明的是，学习编程是一个实践的过程，除了需要看书、查阅资料，亲自动手编写、调试程序才是至关重要的。通过实际的编程和积极的思考，读者可以很快掌握许多宝贵的编程经验，这些编程经验对开发者而言是不可或缺的。

二、本书内容简介

本书可作为Python程序设计课程的教材，也可为具备一定Python编程基础的读者提供学习参考。此外，它还可作为Python应用开发人员的首选参考书。对于那些希望利用业余时间学习一门有趣的编程语言的读者，本书同样适用。

本书共分11章，内容涵盖了Python编程基础、Python语法基础、Python控制语句、Python函数与模块、Python文件操作、面向对象程序设计、Python GUI编程、Python多线程编程技术、Python访问数据库、Python图像处理、Python网络编程等，各章内容简介如下。

章节	内容说明
第1章	介绍Python编程基础，包括语言特点、环境配置、IDLE、基本输入输出、代码规范和获取帮助
第2章	讲解Python语法基础，包括数据类型、变量、运算符和序列数据结构，为编程实践奠定基础
第3章	讲解分支和循环控制语句，包括if、while和for循环等语句，并通过示例阐述常用算法以提高程序设计能力
第4章	介绍函数与模块的基本概念，包括函数定义、参数传递、闭包、递归及模块管理技巧
第5章	探讨文件与文件夹操作，包括打开/关闭文件、使用不同模式进行数据读写等
第6章	深入面向对象设计，重点讲解封装、继承和多态，以及类与对象的定义与管理
第7章	剖析Python图形用户界面(GUI)编程，重点使用Tkinter库创建交互界面，涵盖组件使用和事件处理
第8章	探讨多线程与多进程编程，讲解threading模块与multiprocessing模块的使用方法
第9章	讲解Python数据库访问，重点介绍SQL的基本概念及SQLite3模块的使用
第10章	介绍Pillow库核心模块，包括图像操作、文本与图形添加、滤镜应用及生成水印和二维码
第11章	深入探讨Python网络编程，涵盖计算机网络基础、TCP/UDP编程、网络安全工具及网络爬虫技术

三、本书配套资源及服务

与书中内容同步的案例操作教学资源可供读者随时扫码学习。此外本书免费提供电子课件、完整代码文档和实例源文件，读者可以扫描下方的二维码获取，也可以进入本书信息支持网站(http://www.tupwk.com.cn/downpage)下载。扫描下方的视频二维码可以观看教学视频。

扫一扫，看视频　　扫码推送配套资源到邮箱

本书由哈尔滨理工大学的席亮、黄山学院的毕津滔和黑龙江东方学院的马俊合作编写，其中席亮编写了第1、2、7、10章，毕津滔编写了第3、4、5、6章，马俊编写了第8、9、11章。由于作者水平所限，本书难免有不足之处，欢迎广大读者批评指正。我们的邮箱是992116@qq.com，电话是010-62796045。

编　者

2025年3月

目　录

-contents-

第 1 章　Python编程基础 …… 1

1.1　Python语言简介 …… 2
1.2　安装与配置Python环境 …… 3
1.2.1　安装Python …… 3
1.2.2　运行Python …… 4
1.3　Python开发环境IDLE …… 5
1.3.1　启动IDLE …… 5
1.3.2　使用IDLE创建Python程序 …… 6
1.3.3　IDLE的常用编辑功能 …… 7
1.3.4　在IDLE中运行和调试Python程序 …… 8
1.4　Python基本输入/输出 …… 8
1.4.1　Python基本输入 …… 9
1.4.2　Python基本输出 …… 10
1.5　Python代码规范 …… 10
1.6　Python帮助信息 …… 12
1.6.1　查看内置函数和类型的帮助信息 …… 12
1.6.2　查看模块中的成员函数信息 …… 12
1.6.3　查看整个模块的信息 …… 13
1.7　课后实践 …… 13

第 2 章　Python语法基础 …… 19

2.1　Python数据类型 …… 20
2.1.1　数值类型 …… 20
2.1.2　字符串 …… 20
2.1.3　布尔类型 …… 23
2.1.4　空值 …… 24
2.1.5　Python数据类型转换 …… 24
2.2　变量和常量 …… 25
2.2.1　变量 …… 25
2.2.2　常量 …… 26
2.3　运算符与表达式 …… 27
2.3.1　运算符 …… 27
2.3.2　表达式 …… 34
2.4　序列数据结构 …… 34
2.4.1　列表 …… 34
2.4.2　元组 …… 38
2.4.3　字典 …… 41
2.4.4　集合 …… 45
2.5　课后实践 …… 46

第 3 章　Python控制语句 …… 51

3.1　选择结构 …… 52
3.1.1　if语句 …… 52
3.1.2　if…else语句 …… 52
3.1.3　if…elif…else语句 …… 54
3.1.4　pass语句 …… 56
3.2　循环语句 …… 56
3.2.1　while语句 …… 56
3.2.2　for语句 …… 58
3.2.3　continue和break语句 …… 60
3.2.4　循环嵌套 …… 60
3.3　常用算法 …… 62
3.3.1　累加和累乘 …… 62
3.3.2　求最大数和最小数 …… 62
3.3.3　枚举法 …… 63
3.3.4　递推与迭代 …… 65
3.4　课后实践 …… 67

第 4 章　Python函数与模块 …… 73

4.1　函数的定义和调用 …… 74
4.1.1　函数的定义 …… 74
4.1.2　函数的调用 …… 76
4.1.3　Lambda表达式 …… 78

4.1.4 函数的返回值 …… 80
4.2 函数参数 …… 83
4.2.1 函数形参和实参的区别 …… 83
4.2.2 参数的传递 …… 85
4.2.3 函数参数的类型 …… 88
4.2.4 变量的作用域 …… 97
4.3 闭包和函数的递归调用 …… 99
4.3.1 闭包 …… 99
4.3.2 函数的递归调用 …… 99
4.4 Python内置函数 …… 103
4.4.1 数学运算函数 …… 103
4.4.2 集合操作函数 …… 104
4.4.3 字符串函数 …… 106
4.4.4 反射函数 …… 108
4.4.5 I/O函数 …… 109
4.5 模块 …… 110
4.5.1 import导入模块 …… 110
4.5.2 自定义模块 …… 112
4.5.3 常用标准模块 …… 113
4.6 课后实践 …… 122

第 5 章 Python文件操作 …… 127

5.1 文件的访问 …… 128
5.1.1 打开文件 …… 128
5.1.2 读取文本文件 …… 130
5.1.3 写文本文件 …… 131
5.1.4 文件内移动 …… 133
5.1.5 文件的关闭 …… 134
5.1.6 二进制文件的读/写 …… 135
5.2 文件夹操作 …… 137
5.2.1 当前工作目录 …… 138
5.2.2 目录操作 …… 138
5.2.3 文件操作 …… 140
5.3 课后实践 …… 143

第 6 章 面向对象程序设计 …… 149

6.1 面向对象程序设计基础 …… 150
6.2 类和对象 …… 151
6.2.1 定义和使用类 …… 151
6.2.2 构造函数 …… 152
6.2.3 析构函数 …… 153
6.2.4 实例属性和类属性 …… 153
6.2.5 私有成员与公有成员 …… 156
6.2.6 方法 …… 157
6.3 类的继承和多态 …… 158
6.3.1 类的继承 …… 159
6.3.2 类的多继承 …… 161
6.3.3 方法重写 …… 162
6.3.4 多态 …… 162
6.3.5 运算符重载 …… 163
6.4 课后实践 …… 164

第 7 章 Python GUI编程 …… 169

7.1 Python GUI库 …… 170
7.1.1 创建Windows窗口 …… 170
7.1.2 几何布局管理器 …… 171
7.2 常用Tkinter组件的使用 …… 176
7.2.1 Tkinter组件 …… 176
7.2.2 标准属性 …… 177
7.2.3 标签 …… 178
7.2.4 按钮 …… 179
7.2.5 单行/多行文本框 …… 182
7.2.6 列表框 …… 184
7.2.7 单选按钮和复选框 …… 186
7.2.8 菜单 …… 190
7.2.9 对话框 …… 195
7.2.10 消息窗口 …… 199
7.2.11 框架 …… 200
7.2.12 滚动条 …… 203
7.3 图形绘制 …… 204
7.3.1 Canvas画布组件 …… 205
7.3.2 Canvas画布对象 …… 206
7.4 Tkinter字体 …… 219
7.4.1 通过元素表示字体 …… 219
7.4.2 创建字体 …… 220
7.5 Python事件处理 …… 221

7.5.1 事件类型 …… 221
7.5.2 事件绑定 …… 223
7.5.3 事件处理函数 …… 225
7.6 课后实践 …… 228

第 8 章 Python多线程编程技术 …… 233

8.1 进程和线程 …… 234
8.1.1 进程 …… 234
8.1.2 线程 …… 234
8.1.3 多进程和多线程 …… 234
8.2 多线程编程 …… 235
8.2.1 Python多线程的特殊性 …… 235
8.2.2 使用threading模块进行多线程编程 …… 236
8.3 多进程编程 …… 250
8.3.1 Python多进程编程的特点 …… 250
8.3.2 使用multiprocessing模块进行多进程编程 …… 250
8.4 课后实践 …… 261

第 9 章 Python访问数据库 …… 265

9.1 数据库基础 …… 266
9.1.1 数据库概念 …… 266
9.1.2 关系型数据库 …… 266
9.1.3 数据库和Python接口程序 …… 267
9.2 结构化查询语言(SQL) …… 268
9.2.1 数据表的建立(CREATE TABLE)和删除(DROP) …… 268
9.2.2 查询语句SELCET …… 269
9.2.3 添加记录语句INSERT INTO …… 271
9.2.4 更新语句UPDATE …… 271
9.2.5 删除语句DELETE …… 272
9.3 SQLite数据库简介 …… 272
9.3.1 SQLite3的数据类型 …… 272
9.3.2 SQLite3的函数 …… 273
9.3.3 SQLite3的模块 …… 275
9.4 Python的SQLite3数据库编程 …… 275
9.4.1 访问数据库的步骤 …… 275
9.4.2 创建数据库和表 …… 277
9.4.3 插入、更新和删除记录操作 …… 277
9.4.4 数据库表的查询操作 …… 278
9.5 课后实践 …… 279

第 10 章 Python图像处理 …… 287

10.1 使用Pillow库 …… 288
10.1.1 Image模块 …… 288
10.1.2 ImageDraw模块 …… 292
10.1.3 ImageFont模块 …… 293
10.1.4 ImageFilter模块 …… 293
10.2 为图片添加水印 …… 294
10.3 生成验证码 …… 296
10.4 生成二维码 …… 298
10.5 课后实践 …… 300

第 11 章 Python网络编程 …… 303

11.1 计算机网络基础知识 …… 304
11.2 TCP和UDP编程 …… 305
11.2.1 TCP编程 …… 305
11.2.2 UDP编程 …… 310
11.3 网络嗅探器 …… 312
11.4 端口扫描器 …… 313
11.5 网络爬虫 …… 315
11.5.1 网络爬虫概述 …… 315
11.5.2 网络爬虫分类 …… 315
11.5.3 网络爬虫的基本原理 …… 316
11.5.4 网络爬虫的常用技术 …… 317
11.6 课后实践 …… 320

第1章

Python 编程基础

Python是一门跨平台、开源的解释型高级动态类型编程语言。Python尤其适合初学编程者，因为它能够让初学者将注意力集中在编程的对象和思维方法上，而不必过于担心语法和类型等外部因素。Python易于学习，并拥有丰富的库，使得开发各种应用程序变得高效。本章将介绍Python语言的优缺点、安装方法，以及Python开发环境IDLE的使用。

1.1 Python语言简介

Python的创始人吉多·范罗苏姆(Guido van Rossum)于1989年底发明了Python语言，其被广泛应用于系统管理任务和科学计算，是最受欢迎的编程语言之一。自2004年以来，Python的使用率持续增长，在2017年TIOBE编程语言指数排行榜中位列第四名(前3名为Java、C和C++)。2017年7月，根据IEEE Spectrum发布的编程语言排行榜显示Python已成为全球最受欢迎的编程语言。2024年，Python无论在数据分析、机器学习，还是Web开发领域，都展现出了强大的适应性，受到广大程序员的青睐，再次突破历史记录，稳居编程语言排行榜的前列。

Python支持命令式编程、函数式编程，完全支持面向对象编程，语法简洁清晰，且拥有大量成熟的扩展库，几乎覆盖所有领域的应用开发。许多开源的专业领域工具库都提供了Python的调用接口，例如著名的计算机视觉库OpenCV、三维可视化库VTK和医学图像处理库ITK。此外，Python专用的科学计算扩展库也非常丰富，经典的有NumPy、SciPy和Matplotlib，它们分别为Python提供了快速数组处理、数值运算和绘图功能。因此，Python及其众多扩展库为工程技术和科研人员处理实验数据、制作图表，甚至开发科学计算应用程序提供了理想的开发环境。

Python为开发者提供了一个全面的标准库，涵盖了网络、文件、GUI、数据库、文本等众多领域。使用Python开发时，许多功能可以直接利用现成的库，而无须从头编写。此外，除内置库外，Python还有大量的第三方库可供使用，开发者也可以将自己封装好的代码作为第三方库分享给他人。Python就像胶水一样，可以将用不同编程语言编写的程序无缝结合，充分发挥各种语言和工具的优势，以满足不同应用领域的需求。因此，Python程序通常简单易懂，初学者不仅能够轻松入门，未来深入学习后也能编写复杂的程序。

此外，Python还支持伪编译，可以将源程序转换为字节码，以优化程序性能并提高运行速度，这使得在没有安装Python解释器和相关依赖包的平台上也能运行Python程序。

许多大型网站都是用Python开发的，例如YouTube、Instagram，以及国内的豆瓣等。许多大公司，包括Google、Yahoo、NASA也在大量使用Python。

任何编程语言都有其缺点，Python的主要缺点包括以下几点。

(1) 运行速度慢。与C语言相比，Python的运行速度较慢，因为Python是一种解释型语言。在执行过程中，Python代码会逐行翻译成CPU能够理解的机器码，这一翻译过程非常耗时。而C语言则是在运行前直接编译成机器码，因此运行速度更快。

(2) 代码无法加密。发布Python程序时，实际上就是在发布源代码。这与C语言不同，C语言可以只发布编译后的机器码(例如在Windows上常见的.exe文件)，而无须公开源代码。由于从机器码反推出C语言代码几乎是不可能的，编译型语言在这方面具有一定的安全性。而解释型语言则必须公开源代码。

(3) 缩进带来的困惑。Python使用缩进来区分语句之间的关系，这对于许多初学者来说可能造成困惑。即使是经验丰富的Python程序员，有时也会因为缩进错误而陷入困境。

最常见的问题是混用制表符(tab)和空格，可能导致代码无法正常运行。

这些缺点在一定程度上限制了Python的应用，但它的优点和灵活性仍然使其在许多领域中广受欢迎。

1.2　安装与配置Python环境

因为Python是跨平台的，它可以在Windows、macOS以及各种Linux/UNIX系统上运行。用户在Windows上编写的Python程序，能够直接在Linux上运行。

学习Python编程的第一步是将Python安装到计算机上。安装完成后，将获得Python解释器(负责执行Python程序)、一个命令行交互环境，以及一个简单的集成开发环境(IDE)。

Python至今已有多个版本。但截至目前，主流版本仍为Python 2.x和Python 3.x，这两个版本并不兼容。Python 2.x版本已于2020年1月1日停止更新，Python 2.7被确定为最后一个Python 2.x版本。因此，如果用户正在使用Python 2.x，建议考虑升级到Python 3.x。

1.2.1　安装Python

1. 在 Mac 上安装 Python

若用户使用的是Mac系统，版本在OSX 10.8到10.10之间，那么系统自带的Python版本是2.x。要安装最新的Python 3.x，可以使用以下两种方法。

- 方法一：从Python官方网站(https://www.python.org)下载Python 3.x的安装程序，双击运行并进行安装。
- 方法二：如果已经安装了Homebrew，可以通过命令行直接执行brew install python@3 来完成安装。

2. 在 Linux 上安装 Python

若用户使用的是Linux，并具备一定的系统管理经验，可以参考以下步骤安装Python 3.x。

(1) 下载Python安装包，例如Python-3.5.0b4.tgz。

(2) 使用解压命令解压下载的文件：

```
tar -zxvf Python-3.5.0b4.tgz
```

(3) 切换到解压后的安装目录：

```
cd Python-3.5.0
```

(4) 执行配置命令：

```
./configure
```

(5) 编译安装：

```
make
make install
```

至此，Python的安装已完成。用户可以通过输入以下命令来验证安装是否成功：

```
python
```

如果安装成功，将看到类似以下的提示信息：

```
Python 3.5.0 (#1, Aug 06 2015, 14:04:52)
[GCC 4.1.1 20061130 (Red Hat 4.1.1-43)] on linux2
```

这里需要注意的是，输出内容可能会因安装版本和编译环境的不同而有所变化。

3. 在 Windows 上安装 Python

首先，根据Windows系统版本(64位或32位)从Python官方网站下载相应的Python 3.x安装程序。下载完成后，运行该安装包，打开图1-1所示的安装界面。

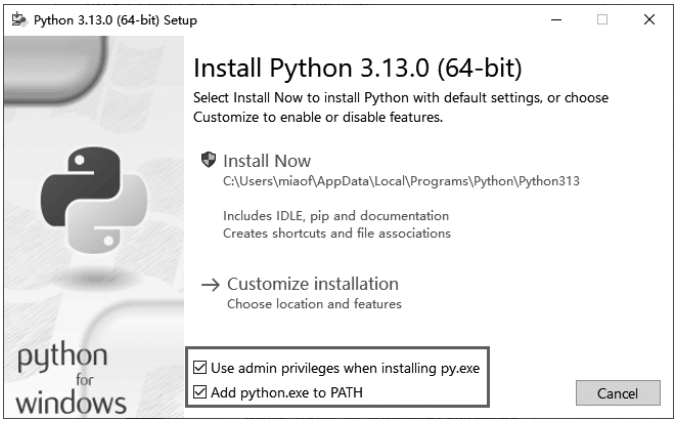

图 1-1　在 Windows 系统中安装 Python

在安装过程中，应在选中Add python.exe to PATH复选框后单击Install Now以完成安装。

1.2.2　运行Python

Python安装成功后，在Windows系统中打开命令提示符(cmd)，执行python命令后，将打开图1-2所示的命令提示符窗口。在该窗口中，如果显示Python的版本信息，说明Python已成功安装。

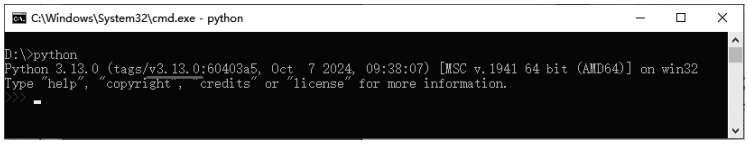

图 1-2　命令提示符窗口

图1-2所示的提示符>>>表示已进入Python交互式环境。在这里，用户可以输入任何Python代码并按回车键，系统会立即返回执行结果。要退出Python交互式环境，可以输入exit()并按回车键，或者直接关闭命令行窗口。

这里需要注意的是，如果出现错误提示：“python不是内部或外部命令，也不是可运行的程序或批处理文件”，这通常是因为Windows按照Path环境变量中设定的路径寻找

python.exe，如果未找到则会报错。如果在安装Python时未在图1-1所示的界面中选中Add python.exe to PATH复选框，用户需要手动将 python.exe 的路径添加到Path环境变量中。如果用户不确定如何修改环境变量，建议重新运行Python安装程序，并确保选中Add python.exe to PATH复选框。

1.3　Python开发环境IDLE

IDLE是Python自带的集成开发环境，提供了一个简单的界面用于编写、调试和运行Python代码。

1.3.1　启动IDLE

在Windows系统中成功安装Python后，可以通过单击系统桌面上的“开始”按钮，在弹出的菜单中选择IDLE(Python 3.x 64-bit)命令来启动IDLE，打开图1-3所示的初始窗口。

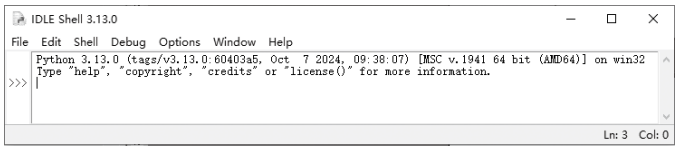

图 1-3　IDLE 的交互式编程模式 (Python Shell)

在图1-3所示的IDLE窗口中，用户可以在其中使用交互式编程模式执行Python命令。使用交互式编程模式时，只需在IDLE提示符>>>后输入相应的命令并按回车键即可执行。如果命令执行成功，将立即看到结果；如果出现问题，则会抛出异常。

例如，查看已安装的Python版本(在IDLE界面的标题栏中也可以直接看到)：

```
>>> import sys
>>> sys.version
```

返回结果可能类似于：'3.13.0 (default, Oct 1 2023, 12:00:00) [MSC v.1926 64 bit (AMD64)]'

此外，还可以进行简单的计算：

```
>>> 3 + 4
```

返回结果为7。

如果尝试执行一个错误的操作，例如：

```
>>> 5/0
```

则会抛出如下异常：

```
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    5/0
ZeroDivisionError: division by zero
```

除了交互式编程模式，IDLE还配备了一个编辑器，用于编写和编辑Python程序(或脚本)文件，并提供调试功能来调试Python脚本。接下来，我们将从IDLE的编辑器开始介绍。

在IDLE界面中，用户通过选择File | New File命令启动编辑器，如图1-4所示。在编辑器中，可以创建一个程序文件，输入代码并将其保存为文件(确保文件扩展名为.py)。

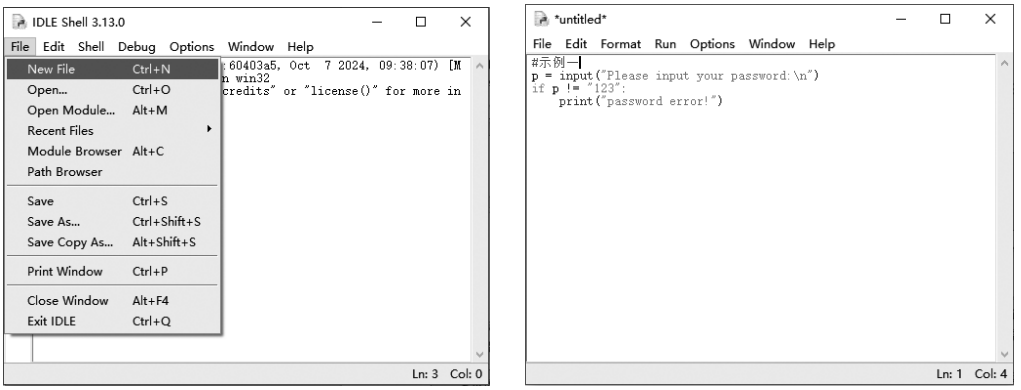

图 1-4　启动 IDLE 的编辑器

1.3.2　使用IDLE创建Python程序

IDLE为开发人员提供了许多实用的特性，如自动缩进、语法高亮显示、单词自动完成和命令历史等。这些功能可以有效提高开发效率。下面通过一个实例来逐一介绍这些特性。示例程序的源代码如下：

```
#示例一
p = input("Please input your password:\n")
if p != "123":
    print("password error!")
```

从图1-4可以看到，不同部分的代码采用了不同的颜色，这就是语法高亮显示。语法高亮显示通过为代码中的不同元素使用不同的颜色，使代码更易于阅读。默认情况下，关键字显示为橙红色，注释为红色，字符串为绿色，解释器的输出为蓝色。在输入代码时，这些颜色会自动应用，语法高亮显示的好处在于可以更容易地区分不同的语法元素，从而提高代码的可读性。此外，语法高亮显示还可以减少出错的可能性。

例如，如果输入的变量名显示为橙红色，说明该名称与保留的关键字冲突，用户需要更改变量名。

单词自动完成的功能允许用户在输入单词的一部分后，从Edit菜单选择Expand Word命令，或直接按下Alt + /组合键来完成该单词。

当在if关键字所在行的冒号后按回车键时，IDLE会自动进行缩进。一般情况下，IDLE将代码缩进一个级别(即4个空格)。如果用户想修改这个默认的缩进量，可以在Format菜单中选择New Indent Width命令进行调整。对于初学者而言，尽管自动缩进功能非常便利，但仍需注意，有时它可能无法完全符合需求，因此建议仔细检查代码的缩进。

创建好程序后，可以在File菜单中选择Save命令来保存程序。如果是新文件，会打开

“另存为”对话框，用户可以在其中指定文件名和保存位置。保存文件后，文件名将自动显示在屏幕顶部的蓝色标题栏中。如果文件中存在未保存的内容，标题栏的文件名前后会出现星号(*)，以提醒用户。

1.3.3　IDLE的常用编辑功能

下面将介绍编写Python程序时常用的IDLE命令，供初学者参考(按菜单分类)。

Edit菜单中常用的命令及其说明如表1-1所示。

表1-1　Edit菜单中的命令及其说明

命　令	说　　明
Undo	撤销上一次的修改
Redo	重复上一次的修改
Cut	将所选文本剪切到剪贴板
Copy	将所选文本复制到剪贴板
Paste	将剪贴板中的文本粘贴到光标所在位置
Find	在窗口中查找特定单词或模式
Find in Files	在指定文件中查找特定单词或模式
Replace	替换特定单词或模式
Go to Line	将光标定位到指定的行首

Format菜单中常用的命令及其说明如表1-2所示。

表1-2　Format菜单中的命令及其说明

命　令	说　　明
Indent Region	将所选内容右移一级，即增加缩进
Dedent Region	将所选内容左移一级，即减少缩进
Comment Out Region	将所选内容注释掉
Uncomment Region	去除所选内容每行前面的注释符
New Indent Width	重新设定缩进宽度，范围为2～16，宽度为2相当于1个空格
Expand Word	单词自动完成
Toggle Tabs	打开或关闭制表位

1.3.4　在IDLE中运行和调试Python程序

1. 运行 Python 程序

要在IDLE中执行Python程序，可以在Run菜单中选择Run Module命令(或直接按 F5 键)。此命令的功能是执行当前打开的文件。例如，在执行示例程序时，用户输入的密码为88，但由于输入错误，输出结果为“password error！”，如图1-5所示。

2. 使用 IDLE 调试器

在软件开发过程中，难免会遇到各种错误，包括语法错误和逻辑错误。对于语法错误，Python 解释器能够轻松检测，并会停止程序运行，而且会给出相应的错误提示。然而，对于逻辑错误，解释器则无能为力，程序可能会继续执行，但运行结果却是错误的。因此，调试程序通常是必要的，以确保代码的正确性和可靠性。

最简单的调试方法是直接显示程序的数据。例如，可以在关键位置使用print语句来输出变量的值，从而确定是否存在错误。然而，这种方法相对烦琐，因为开发人员必须在所有可疑的地方插入打印语句，并且在调试完成后，还需要手动删除这些打印语句。

除使用print语句外，还可以利用调试器进行更有效的调试。调试器可以帮助分析被调试程序的数据，并监视程序的执行流程。其功能包括暂停程序执行、检查和修改变量、调用方法而不更改程序代码等。IDLE 也提供了一个调试器，帮助开发人员查找逻辑错误。以下是 IDLE 调试器的简单使用方法。

在IDLE Shell窗口中，选择Debug菜单中的Debugger命令，即可启动IDLE的交互式调试器。这时，IDLE 会打开如图 1-6 所示的Debug Control窗口，并在窗口中输出[DEBUG ON]，后面显示>>>提示符。

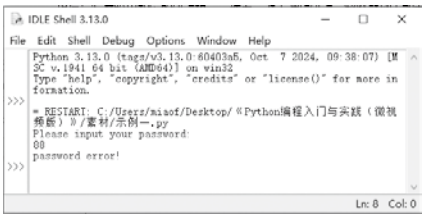

图 1-5　程序运行界面

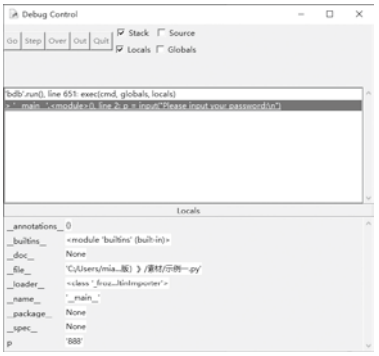

图 1-6　Debug Control 调试窗口

此时，用户可以像平常一样使用 IDLE Shell 窗口，只不过现在输入的任何命令都是在调试器下执行的。

1.4　Python基本输入/输出

Python的基本输入通过input()函数获取用户输入，而输出则使用print()函数将信息显示在控制台。

1.4.1　Python基本输入

在 Python中进行程序设计时，输入是通过input()函数实现的。input()函数的一般格式为：

```
x = input("输入一个值：")
```

该函数返回用户输入的对象，可以输入数字、字符串和其他任意类型的对象。不过，Python 2.7 和 Python 3.x的input()函数虽然形式相同，但在解释上略有不同。

在 Python 2.7 中，返回结果的类型由输入值时所使用的界定符决定。例如，以下是Python 2.7 的示例代码：

```
x = input("Please input:")
```

如果输入为：

```
Please input: 3
```

由于没有使用界定符，结果会被解释为整数：

```
>>> print type(x)
<type 'int'>
```

如果输入为：

```
Please input: '3'
```

由于使用了单引号，结果将被解释为字符串：

```
>>> print type(x)
<type 'str'>
```

在 Python 3.x中，input()函数始终返回字符串类型，因此不再区分输入值的类型。

在 Python 2.7中，还有一个内置函数raw_input()，用于接收用户输入的值。与 input()函数不同，raw_input()函数始终返回字符串类型，无论用户使用什么界定符。

在Python 3.x中，不再提供raw_input()函数，取而代之的是使用input()函数来接收用户的键盘输入。在Python 3.x中，input()函数的返回结果也始终是字符串，不论用户输入的数据使用何种界定符。因此，用户需要将输入的字符串转换为相应的类型进行处理，这与Python 2.7 中的raw_input()函数相似。

以下是Python 3.x的示例代码：

```
x = input('Please input:')
```

如果输入为：

```
Please input: 3
```

则输出结果为：

```
>>> print(type(x))
<class 'str'>
```

如果输入为：

```
Please input: '1'
```

输出结果同样为：

```
>>> print(type(x))
<class 'str'>
```

如果输入为：

```
Please input: [1, 2, 3]
```

输出结果仍然是：

```
>>> print(type(x))
<class 'str'>
```

因此，在 Python 3.x中，所有用户输入的内容都将以字符串形式返回。

1.4.2 Python基本输出

Python 2.7和Python 3.x的输出方法并不完全一致。在Python 2.7中，使用print语句进行输出，而在Python 3.x中，则使用print()函数。此外，一个重要的区别是，在Python 2.7中，如果在print语句后加上逗号(,)，则表示输出内容后不换行。例如：

```
for i in range(10):
    print i,
```

输出结果是：0123456789。

在 Python 3.x 中，为了实现类似的功能，可以使用以下方法：

```
for i in range(10, 20):
    print(i, end=' ')
```

输出结果是：10 11 12 13 14 15 16 17 18 19。

1.5 Python代码规范

Python 代码规范(PEP 8)强调代码的可读性，包括使用4个空格进行缩进、适当命名、保持行长度不超过 79 个字符，以及在逻辑块之间添加空行等。

(1) 缩进。Python 程序通过代码块的缩进来体现逻辑关系，缩进的结束标志着一个代码块的结束。类定义、函数定义、选择结构和循环结构等都以行尾的冒号表示缩进的开始。同一层级的代码块必须保持一致的缩进量。例如：

```
for i in range(10):                    # 循环输出0～9的数字
    print(i, end=', ')
```

一般而言，建议使用4个空格作为基本缩进单位，避免使用制表符(tab)。在 IDLE 开发

环境中，可以通过以下操作来进行代码块的缩进和反缩进：选择 Format | Indent Region(或Dedent Region)命令。

(2) 注释。一个好的、可读性强的程序通常包含超过 20% 的注释。常用的注释方式主要有以下两种。

- 方法一：使用#开始，表示本行#之后的内容为注释。例如：

```
# 循环输出0～9的数字
for i in range(10):
    print(i, end=' ')
```

- 方法二：用三重引号"""或'''包围的内容，如果不属于任何语句，则被解释器视为注释。例如：

```
"""循环输出0～9的数字，可以包含多行文字"""
for i in range(10):
    print(i, end=' ')
```

在 IDLE 开发环境中，可以通过以下操作快速注释或解除注释一段代码：选择 Format | Comment Out Region(或Uncomment Region)命令。

(3) 每个import语句应该只导入一个模块，而不应该一次导入多个模块。例如：

```
>>> import math                          # 导入math数学模块
>>> math.sin(0.5)                        # 求0.5的正弦
>>> import random                        # 导入random随机模块
>>> x = random.random()                  # 获得[0, 1)内的随机小数
>>> y = random.random()
>>> n = random.randint(1, 100)           # 获得[1, 100]范围内的随机整数
```

虽然可以一次导入多个模块，但不提倡这样做。导入的顺序应为：首先导入Python内置模块，其次导入第三方模块，最后导入自己开发的项目中的其他模块。避免使用from module import *，除非是导入常量定义模块或其他确保不会出现命名冲突的模块。

(4) 如果一行语句过长，可以在行尾添加反斜杠“\”来换行，但更推荐使用圆括号将多行内容包含起来。例如：

```
x = '这是一个非常长非常长非常长非常长\
    非常长非常长非常长非常长非常长的字符串'          # 使用反斜杠"\"换行
x = ('这是一个非常长非常长非常长非常长'
    '非常长非常长非常长非常长非常长的字符串')        # 圆括号中的行会连接起来
```

另一个示例：

```
if (width == 0 and height == 0 and
        color == 'red' and emphasis == 'strong'):     # 圆括号中的行会连接起来
    y = '正确'
else:
    y = '错误'
```

(5) 必要的空格与空行。建议在运算符两侧、函数参数之间及逗号两侧使用空格分隔。此外，不同功能的代码块之间、不同的函数定义之间应该增加一个空行，以提高可读性。

(6) 常量命名。常量名应全部用大写字母，并用下画线连接各个单词。类名的首字母应大写。例如：

```
CONSTANT_VALUE = 0
THIS_IS_A_CONSTANT = 1
```

1.6 Python帮助信息

使用Python的帮助功能对于学习和开发都是非常重要的。在Python中，可以使用help()方法来获取帮助信息。其使用格式如下：

```
help(对象)
```

以下将分为三种情况进行说明。

1.6.1 查看内置函数和类型的帮助信息

在IDLE环境中输入以下命令，可以查看内置 max 函数的帮助信息(如图1-7所示)：

```
>>> help(max)
>>> help(list)                    # 可以获取list列表类型的成员方法
>>> help(tuple)                   # 可以获取tuple元组类型的成员方法
```

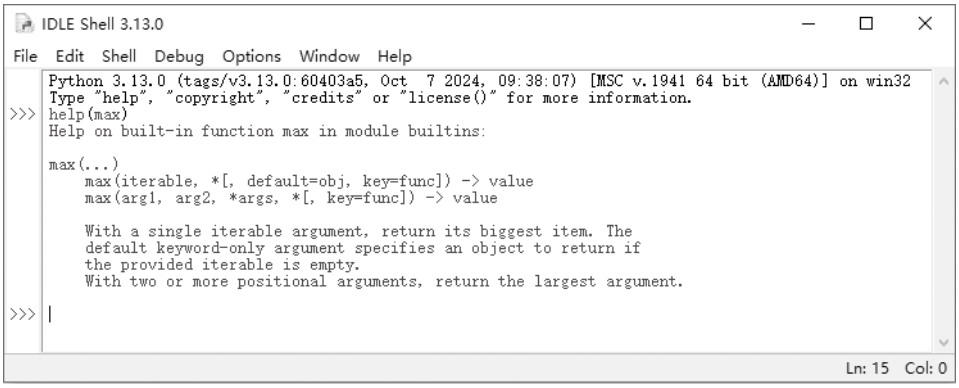

图 1-7 查看内置 max 函数的帮助信息

1.6.2 查看模块中的成员函数信息

要查看某个模块中的成员函数信息，可以使用以下命令：

```
>>> import os
>>> help(os.fdopen)
```

上述例子查看os模块中的fdopen成员函数的信息，得到如图1-8所示的提示信息：

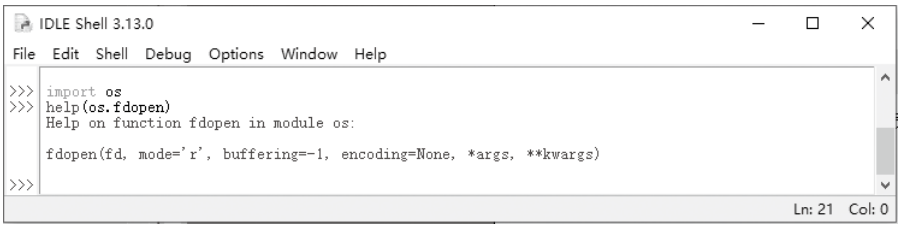

图 1-8　查看 os 模块中的 fdopen 成员函数信息

1.6.3　查看整个模块的信息

使用help(模块名)可以查看整个模块的帮助信息。这里需要注意的是，首先需要使用import导入该模块。例如，要查看math模块的方法：

```
>>> import math
>>> help(math)
```

帮助信息如图 1-9所示。

要查看 Python 中所有的模块，可以使用以下命令：

```
help("modules")
```

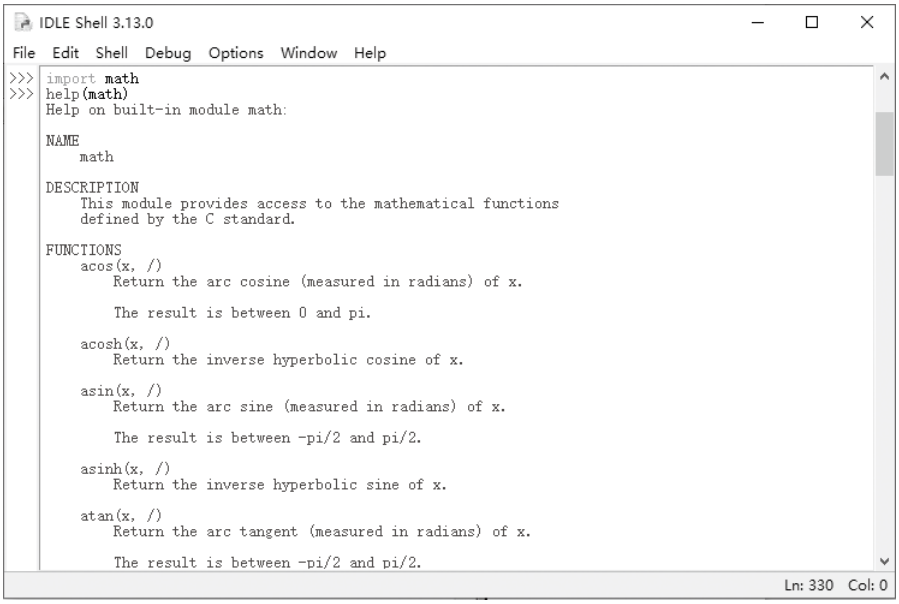

图 1-9　查看 math 模块的帮助信息

1.7　课后实践

本章主要介绍了Python编程的基础知识，包括Python语言的特点与应用、环境的安

装与配置、IDLE开发环境的使用、基本输入输出操作、代码规范以及获取帮助信息的方式。首先，简要概述了Python的易读性和强大的库支持，使其成为初学者和专业开发者的理想选择。接着，详细说明了在macOS、Linux和Windows等操作系统上安装Python的步骤，以及如何运行Python程序。随后，介绍了IDLE作为集成开发环境的使用方法，包括创建、编辑和调试程序的基本操作。接下来，讲解了如何使用input()和print()函数进行基本的输入输出，实现简单的交互式程序。此外，还强调了遵循PEP 8代码规范的重要性，以提高代码的可读性和可维护性。最后，本章提供了获取Python帮助信息的途径，帮助读者在编程过程中解决问题。

本章为用户进一步学习Python奠定了扎实的基础。在下面的课后实践部分，用户可以通过动手实现几个简单的编程实例并完成课后的思考练习，加深对Python的了解。

一、应用实例

【例1-1】熟悉IDLE开发环境。

本例将使用标准的IDLE作为开发环境，帮助用户熟悉Python的强大功能。有时，我们可能需要同时安装多个不同版本的Python，并根据不同的开发需求在这多个版本之间切换。多个版本并存不会影响在IDLE环境中直接运行程序，只需启动相应版本的IDLE即可。

熟练掌握开发环境提供的一些快捷键可以显著提升开发效率。在IDLE环境中，除了常用的撤销(Ctrl+Z)、全选(Ctrl+A)、复制(Ctrl+C)、粘贴(Ctrl+V)和剪切(Ctrl+X)等快捷键，还有一些其他快捷键，如表1-3所示。

表1-3　IDLE中的常用快捷键

快 捷 键	说　　明
Tab	补全单词，列出所有可选单词供选择
Alt + P	浏览历史命令(上一条)
Alt + N	浏览历史命令(下一条)
Ctrl + F6	重启Shell，之前定义的对象和导入的模块全部失效
F1	打开Python帮助文档
Alt+/	自动补全之前出现过的单词，若有多个单词具有相同前缀，则在多个单词间循环切换
Ctrl+]	代码块缩进
Ctrl+\	取消代码块缩进
Alt+3	注释代码块
Alt+4	取消代码块注释

(1) 进入IDLE开发环境，直接在Python提示符>>>后输入以下命令并按Enter键执行。执行成功后，会立即显示结果；如果出现错误，则会提示错误信息并抛出异常。

```
>>>2+1
3
>>>import math                    # 导入Python标准库math
>>>math.sqrt(25)                  # 使用标准库函数sqrt计算平方根
5.0
>>>3**2                           # 使用**计算幂运算
9
>>> 3 * (1 + 9)
30
>>>2/0                            # 除以0错误，会抛出异常
Traceback (most recent call last):
File "<pyshell#5>", line 1, in <module>
2/0
ZeroDivisionError: division by zero
>>>x = 'hello                     # 语法错误，字符串的末尾缺少一个单引号
SyntaxError: EOL while scanning string literal
```

从以上代码可以看出，交互模式一般用于实现简单的业务逻辑或验证某些功能。复杂的业务逻辑通常通过编写Python程序来实现，这样可以更方便地不断完善和复用代码。

(2) 在IDLE界面中选择File | New File 命令创建一个程序文件，输入代码并保存为文件(确保扩展名为.py；如果是GUI程序，扩展名为.pyw)。

```
def main():
    print("this is a test program")

main()
```

(3) 使用Run | Run Module菜单命令运行程序，程序的运行结果将直接显示在IDLE的交互界面中。例如，以上程序输出结果如下：

```
==== RESTART: C:/Users/miaof/AppData/Local/Programs/Python/Python313/test.py ===
this is a test program
```

【例1-2】熟悉pip工具。

Python语言中有三类库：内置库、标准库和扩展库。其中，内置库和标准库在成功安装Python后即随之安装。内置库可以直接使用，无须通过import命令导入；而标准库和扩展库则需要先导入才能使用。扩展库主要通过pip工具进行管理。

在使用pip工具之前，需要确认其是否可用。可以按下Ctrl+R键，在打开的“运行”对话框的“打开”文本框中输入cmd后单击“确定”按钮(如图1-10所示)，进入命令窗口，输入pip命令，如图1-11所示(如果pip工具无法使用，可以检查Python的安装目录，找到安装目录中的pip.exe文件，并将其添加到系统环境变量Path中。完成后，重启命令提示符并重试)。

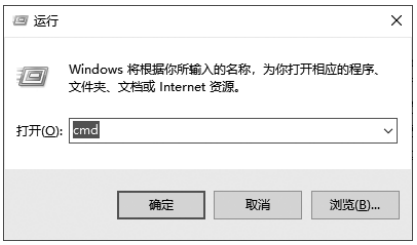

图 1-10 “运行”对话框

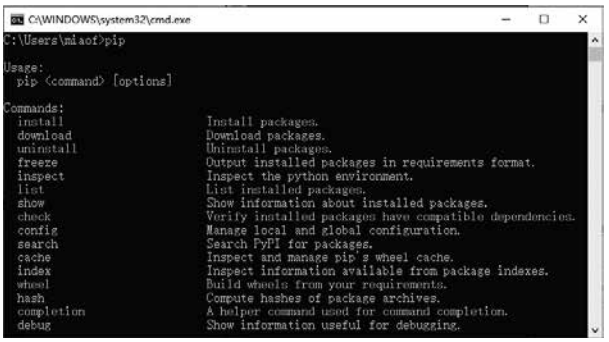

图 1-11 检查 pip 工具

常用的pip命令如下。

- pip list：查看已安装的扩展库。
- pip install package_name：安装名为package_name的扩展库。
- pip uninstall package_name：卸载名为package_name的扩展库。

例如，执行以下命令安装pandas库：

```
pip install pandas
```

【例1-3】编写程序，求解30° 的正弦函数值

分析：我们知道正弦函数用sin表示，但不确定在Python中是否有特殊的要求，因此可以在IDLE环境中先使用帮助命令help进行查询。

```
>>> import math
>>> help(math.sin)
Help on built-in function sin in module math:

sin(x, /)
    Return the sine of x (measured in radians).
```

通过帮助信息，我们了解到sin函数的参数x是以弧度(radians)为单位的。接下来，可以查看math模块中是否有将角度转换为弧度的函数。

```
>>>help(math)
Help on built-in module math:

NAME
    math

DESCRIPTION
    This module provides access to the mathematical functions
    defined by the C standard.

FUNCTIONS
    acos(x, /)
```

```
            Return the arc cosine (measured in radians) of x.

            The result is between 0 and pi.

    acosh(x, /)
            Return the inverse hyperbolic cosine of x.

……
    radians(x, /)
            Convert angle x from degrees to radians.

    remainder(x, y, /)
            Difference between x and the closest integer multiple of y.
……
DATA
        e = 2.718281828459045
        inf = inf
        nan = nan
        pi = 3.141592653589793
        tau = 6.283185307179586

FILE
        (built-in)
```

其中，radians函数的功能是将角度转换为弧度，而degrees函数则用于将弧度转换为角度。下面运用三种模块的导入方法来求解30°的正弦函数值。

(1) 使用Windows系统自带的记事本工具编写程序。

程序代码1：

```
import math
x = math.sin(math.radians(30))
print(x)
```

程序代码2：

```
from math import *
x = sin(radians(30))
print(x)
```

程序代码3：

```
from math import sin, radians
x = sin(radians(30))
print(x)
```

(2) 将以上代码分别保存为python_1.py、python_2.py和python_3.py。

(3) 在保存.py文件的文件夹地址栏中输入cmd，按下Enter键，打开命令行窗口，然后在命令行中输入以下命令运行python_1.py：

```
python python_1.py
```

输出结果：

```
0.49999999999999994
```

使用同样的方法，在命令行中运行python_2.py和python_3.py，输出结果同上。

二、思考练习

1. 简要说明如何选择适合的Python版本。
2. Python语言有哪些优点和缺点？
3. Python的基本输入/输出函数是什么？
4. 如何在IDLE中运行和调试Python程序？
5. 为什么在程序中加入注释是重要的？如何在程序中添加注释？
6. 编写程序，让用户输入一个三位以上的整数，并输出其百位以上的数字。例如，用户输入1234，则程序输出12(提示：使用整除运算)。

第 2 章

Python 语法基础

数据类型是程序中最基本的概念，确定数据类型后，才能明确变量的存储方式及其操作。表达式则是用于表示计算求值的式子。数据类型和表达式是程序员编写程序的基础。因此，本章介绍的内容构成了进行 Python 程序设计的基本框架。

2.1 Python数据类型

计算机程序能够处理各种数值类型，但不仅限于数值。计算机还可以处理文本、图形、音频、视频、网页等多种数据，因此需要定义不同的数据类型。

2.1.1 数值类型

Python 中的数值类型用于存储数值。Python 支持4种不同的数值类型，如表2-1所示。

表2-1 Python支持的数值类型

数 值 类 型	说 明
整型(int)	通常称为整数，表示正整数、负整数或零，不带小数点
长整型(long)	表示无限大小的整数，通常在整数末尾加上大写或小写字母L。在Python 3.x中，仅存在一种整数类型int，不再有Python 2.x中的long类型
浮点型(float)	浮点型由整数部分和小数部分组成，也可以使用科学记数法表示(例如，2.78e2 表示$2.78\times10^2=278$)
复数(complex)	复数由实数部分和虚数部分构成，可以用a + bj或complex(a, b)表示，其中虚部用字母j表示。例如，2 + 3j

需要注意的是，数值类型是不可变的，这意味着如果要改变数值数据类型的值，将会重新分配内存空间。

2.1.2 字符串

字符串是Python中最常用的数据类型。可以使用引号来创建字符串。Python不支持单独的字符类型，单个字符在Python中也作为字符串处理。Python使用单引号和双引号表示字符串是等效的。

1. 创建和访问字符串

创建字符串非常简单，只需为变量分配一个值即可。例如：

```
var1 = 'Hello World!'
var2 = "Python Programming"
```

在Python中，可以使用方括号访问子字符串。例如：

```
var1 = "Hello, World!"
var2 = "Python Programming"
print("var1[0]:", var1[0])          # 获取索引为0的字符，注意索引从0开始
print("var2[1:5]:", var2[1:5])      # 切片操作
```

上述代码的执行结果为：

```
var1[0]: H
var2[1:5]: ytho
```

切片是对字符串(或其他序列)的一种操作，格式为字符串后跟方括号，方括号中包含一对可选的数字，用冒号分隔，例如 [1:5]。切片操作中的第一个数字(冒号之前)表示切片的起始位置，第二个数字(冒号之后)表示切片的结束位置。

- 如果不指定起始位置，Python 将默认从字符串的开头开始切片。
- 如果不指定结束位置，Python 将默认切片到字符串的末尾。

需要注意的是，切片返回的内容从起始位置开始，正好在结束位置之前结束。例如，[1:5] 表示获取从第二个字符到第六个字符之前(即第5个字符)的部分。

2. Python 转义字符

在Python中，当需要在字符串中使用特殊字符时，可以使用反斜杠“\”进行转义。表 2-1 所示为常用的转义字符。

表2-2　转义字符

转义字符	说　明	转义字符	说　明
\(在行尾时)	续行符	\r	回车
\\	反斜杠	\a	响铃
\n	换行	\f	换页
\t	横向制表符	\b	退格(Backspace)
\v	纵向制表符	\000	空
\'	单引号	\xyy	十六进制数，yy代表的字符
\"	双引号	\oyy	八进制数，yy代表的字符

3. Python 字符串运算符

Python中的字符串运算符如表2-3所示。在以下示例中，变量a的值为字符串 "Hello"，变量b的值为字符串"Python"。

表2-3　Python字符串运算符

运算符	说　明	示　例
+	字符串连接	a+b输出结果：HelloPython
*	重复输出字符串	a * 2输出结果：HelloHello

(续表)

运 算 符	说　　明	示　　例
[ ] [:]	通过索引获取字符串中的字符 截取字符串中的一部分	a[1]输出结果：e a[1:4]输出结果：ell
in	成员运算符，如果字符串中包含给定的字符返回True	'H' in a输出结果：True
not in	成员运算符。用于判断字符串中是否不包含给定的字符。如果字符串中不包含该字符，则返回True	'M' not in a输出结果：True

4. 字符串格式化

Python支持格式化字符串的输出。虽然格式化字符串可能涉及复杂的表达式，但最基本的用法是将一个值插入到包含字符串格式化符的模板中。在Python中，字符串格式化使用与 C 语言中 printf 函数相同的语法。例如：

```
print("我的名字是 %s，年龄是 %d" % ('dsm', 45))
```

在这个例子中，Python使用一个元组将多个值传递给模板，每个值对应一个字符串格式符。上述代码将'dsm'插入到%s处，将45插入到%d处。因此输出结果为：

```
我的名字是 dsm，年龄是 45
```

Python 中的字符串格式化符如表2-4所示。

表2-4　Python字符串格式化符

符 号	说　　明	符 号	说　　明
%c	格式化字符	%u	格式化无符号整型
%f	格式化浮点数字，可指定小数点后的精度	%g	%f和%e的简写
%s	格式化字符串	%G	%f和%E的简写
%e	用科学记数法格式化浮点数	%o	格式化八进制数
%d	格式化十进制整数	%x	格式化十六进制数
%E	与%e相同，用科学记数法格式化浮点数	%X	格式化十六进制数(大写)

字符串格式化示例：

```
charA = 65
charB = 66
print("ASCII 码 65 代表：%c" % charA)
```

```
print("ASCII 码 66 代表：%c" % charB)
Num1 = 0xFE
Num2 = 0xAB03
print('转换成十进制分别为：%d和%d' % (Num1, Num2))
Num3 = 1200000
print('转换成科学记数法为：%e' % Num3)
Num4 = 65
print('转换成字符为：%c' % Num4)
```

输出结果：

```
ASCII 码 65 代表：A
ASCII 码 66 代表：B
转换成十进制分别为：255和43779
转换成科学记数法为：1.200000e+06
转换成字符为：A
```

2.1.3　布尔类型

Python 支持布尔类型的数据，布尔类型只有两个值：True和False。布尔类型支持以下几种运算。

- and(与运算)：只有当两个布尔值都为True时，计算结果才为True。

```
True and True          # 结果是True
True and False         # 结果是False
False and True         # 结果是False
False and False        # 结果是False
```

- or(或运算)：只要有一个布尔值为True，计算结果就是True。

```
True or True           # 结果是True
True or False          # 结果是True
False or True          # 结果是True
False or False         # 结果是False
```

- not(非运算)：将True变为False，或者将False变为True。

```
not True               # 结果是False
not False              # 结果是True
```

布尔运算在计算机中用于条件判断，根据计算结果为True或False，计算机可以自动执行不同的后续代码。

在Python中，布尔类型还可以与其他数据类型进行and、or 和not运算。在这些运算中，以下几种情况被认为是 False：

- 为0的数字(包括0和0.0)；
- 空字符串("")；

- 表示空值的None；
- 空集合、空元组( )、空列表[ ]、空字典{ }。

其他的值都被视为True。例如：

```
a = 'python'
print(a and True)                    # 结果是True
b = ' '
print(b or False)                    # 结果是False
```

2.1.4 空值

空值是Python中一个特殊的值，用None表示。它不支持任何运算，也没有任何内置函数方法。None与任何其他数据类型进行比较时，永远返回False。在Python中，未指定返回值的函数会自动返回None。

2.1.5 Python数据类型转换

Python提供了多种数据类型转换函数，如表 2-5所示。

表2-5 数据类型转换函数

数据类型转换函数	说　明
int(x[, base])	将x转换为一个整数
long(x[, base])	将x转换为一个长整数(Python 3.x中已移除)
float(x)	将x转换为一个浮点数
complex(real[, imag])	创建一个复数
str(x)	将对象x转换为字符串
repr(x)	将对象x转换为表达式字符串
eval(str)	计算字符串中的有效Python表达式，并返回一个对象
tuple(s)	将序列s转换为一个元组
list(s)	将序列s转换为一个列表
chr(x)	将一个整数(ASCII或Unicode编码)转换为一个字符
ord(x)	将一个字符转换为它的ASCII整数值(汉字为Unicode编码)
bin(x)	将整数x转换为二进制字符串，例如 bin(24)结果是'0b11000'
oct(x)	将一个数字转换为八进制，例如oct(24)结果是'0o30'
hex(x)	将整数x转换为十六进制字符串，例如hex(24)结果是'0x18'
chr(i)	返回整数i对应的ASCII字符，例如chr(65)结果是'A'

示例：

```
x = 20                          #十进制为20
y = 345.6
print(oct(x))                   # 打印结果是'0o24'
print(int(y))                   # 打印结果是345
print(float(x))                 # 打印结果是20.0
print(chr(65))                  # 'A'对应的ASCII为65，打印结果是'A'
print(ord('B'))                 # 'B'的ASCII为66，打印结果是66
print(ord('中'))                # '中'的Unicode为20013，打印结果是20013
print(chr(20018))               # '串'的Unicode为20018，打印结果是'串'
```

2.2　变量和常量

Python 的变量是程序中对数据的命名引用，用于存储和操作数据，其值可以在程序运行时改变，而常量则是指在程序中其值不可改变的固定数据。

2.2.1　变量

变量的概念基本上与初中代数中的方程变量相似，但在计算机程序中，变量不仅可以表示数字，还可以表示任意数据类型。在程序中，变量通过一个变量名来表示，变量名必须由大小写字母、数字和下画线的组合构成，并且不能以数字开头。例如：

```
a = 1                           # 变量a是一个整数
t_007 = "T007"                  # 变量t_007是一个字符串
Answer = True                   # 变量Answer是一个布尔值True
```

在Python中，等号“=”用于赋值，可以将任意数据类型赋值给变量，同一个变量可以反复赋值，并且可以赋值不同类型的数据。例如：

```
a = 123                         # a是整数
a = 'ABC'                       # a变为字符串
```

这种变量类型不固定的语言称为动态语言，与之对应的是静态语言。静态语言在定义变量时必须指定变量类型，如果赋值时类型不匹配，则会报错。例如，C 语言是静态语言，赋值语句如下(//表示注释)：

```
int a = 123;                    // a是整数类型变量
a = "ABC";                      // 错误，不能将字符串赋值给整型变量
```

与静态语言相比，动态语言更灵活，正是这个原因。

需要注意的是，赋值语句中的等号与数学中的等号并不相同。例如，在以下代码中：

```
x = 10
x = x + 2
```

如果从数学角度理解x = x + 2，这显然是不成立的。但在程序中，赋值语句会先计算右侧的表达式x + 2，得到结果12，然后将其赋值给变量x。由于x之前的值是10，重新赋值后，x的值变为12。

理解变量在计算机内存中的表示也非常重要。当执行以下代码时：

```
a = 'ABC'
```

Python 解释器会进行以下两件事情：

在内存中创建了一个字符串'ABC'。

在内存中创建了一个名为a的变量，并将其指向字符串'ABC'。

也可以将一个变量a的值赋给另一个变量b，这个操作实际上是让变量b指向变量 a 所指向的数据。例如，下面的代码：

```
a = 'ABC'
b = a
a = 'XYZ'
print(b)                                # 输出'ABC'
```

最后一行打印出变量b的内容到底是'ABC'还是'XYZ'呢？如果从数学的角度理解，可能会错误地认为b和a是相同的，因此也应该是'XYZ'。但实际上，b的值是'ABC'。下面逐行执行代码，看看到底发生了什么：

(1) 执行a = 'ABC'，Python解释器创建了字符串'ABC'和变量a，并让a指向这个字符串。

(2) 执行b = a，解释器创建了变量b，并让b指向a所指向的字符串'ABC'。

(3) 执行a = 'XYZ'，解释器创建了字符串'XYZ'，并将a的指向更改为'XYZ'，但b的指向未受影响。

因此，最后打印变量b的结果自然是 'ABC'。

当变量不再需要时，Python会自动回收其占用的内存空间。此外，也可以使用del语句手动删除某些变量。del语句的语法如下：

```
del var1[, var2[, var3[..., varN]]]
```

可以通过del语句删除单个或多个变量对象，例如：

```
del a                                   # 删除单个变量对象
del a, b                                # 删除多个变量对象
```

2.2.2 常量

常量是指不可更改的变量，例如常用的数学常数 π 就是一个常量。在Python中，常量通常使用全大写的变量名表示，例如：

```
PI = 3.1415926535
```

然而，实际上PI仍然是一个变量，Python并没有任何机制来保证PI的值不会被改变。

因此，虽然使用全大写的变量名来表示常量是一种约定，但在实际操作中，PI的值是可以被修改的。

2.3　运算符与表达式

在程序中，表达式用于计算和求值，它由运算符(操作符)和运算数(操作数)组成。运算符是表示进行某种运算的符号，而运算数则可以是常量、变量和函数等。例如，在表达式4 ＋ 5中，4和5被称为操作数，＋则是运算符。

2.3.1　运算符

Python支持多种类型的运算符，包括：算术运算符、比较(关系)运算符、逻辑运算符、赋值运算符、位运算符、成员运算符和标识运算符。

1. 算术运算符

算术运算符用于执行数学运算，Python中的算术运算符如表2-6所示。假设变量a = 10和变量 b = 20。

表2-6　Python中的算术运算符

运 算 符	说　　明	示　　例
＋	加法	a＋b = 30
－	减法	a － b =－10
*	乘法	a * b = 200
/	除法	b / a = 2
%	模运算符(求余)	b % a =0，7 % 3 = 1
**	指数运算(幂)	a ** b = 10^{20} (10的20次方)
//	整除，结果为商的小数点后部分舍去的整数	9 // 2 = 4而9.0 // 2.0 = 4.0

需要注意以下几点：

(1) 乘号不可省略。在Python中，算术表达式的乘号(*)不能省略。例如，数学表达式$b^2 - 4ac$应该写成：b * b － 4 * a * c。

(2) 字符集限制。在 Python 表达式中，只能使用字符集允许的字符。例如，数学表达式πr^2应该写成：math.pi * r * r，其中math.pi是Python已定义的模块变量。示例代码如下：

```
>>> import math
>>> math.pi
```

结果为 3.141592653589793

(3) 使用圆括号控制运算优先级。在Python中，算术表达式只能使用圆括号来改变运算的优先顺序，不能使用{ }或[ ]。可以使用多层圆括号，且左右括号必须配对，运算时从内层括号开始，由内向外依次计算表达式的值。

2. 关系运算符

关系运算符用于比较两个值，运算结果为True(真)或False(假)。Python中的关系运算符如表2-7所示。假设变量a = 10和变量b = 20。

表2-7　Python中的关系运算符

运 算 符	说　　明	示　　例
==	检查两个操作数的值是否相等，如果相等则结果为True	(a == b)为False
!=	检查两个操作数的值是否不相等，如果不相等则结果为True	(a != b)为True
<>	检查两个操作数的值是否不相等(此运算符在Python 3.x中已被废弃，使用!=替代)	(a <> b)为True
>	检查左操作数的值是否大于右操作数的值，如果是则结果为 True	(a > b)为False
<	检查左操作数的值是否小于右操作数的值，如果是则结果为 True	(a < b)为True
>=	检查左操作数的值是否大于或等于右操作数的值，如果是则结果为True	(a >= b)为False
<=	检查左操作数的值是否小于或等于右操作数的值，如果是则结果为True	(a <= b)为True

需要注意的是，关系运算符的优先级低于算术运算符。例如，表达式a + b > c等效于(a + b) > c。

3. 逻辑运算符

Python 提供了三种逻辑运算符，分别如下。

- and：逻辑与，二元运算符。
- or：逻辑或，二元运算符。
- not：逻辑非，一元运算符。

这三种逻辑运算符的含义如下(设a和b是两个参与运算的逻辑量)。

- a and b的含义是，当a和b均为真时，表达式的值为真(True)；否则为假(False)。
- a or b的含义是，当a和b中至少有一个为真时，表达式的值为真；如果a和b均为假，则表达式的值为假。
- not a的含义是，当a为假时，表达式的值为真；否则为假。

Python中的逻辑运算符如表2-8所示。

表2-8　Python中的逻辑运算符

运 算 符	说　　明	示　　例
and	逻辑与运算符。如果两个操作数都为真(非零)，则结果为真(True)	(True and True)为True
or	逻辑或运算符。如果至少有一个操作数为真(非零)，则结果为真	(True or False)为True
not	逻辑非运算符。用于反转操作数的逻辑状态。如果操作数为真，则返回假(False)；否则返回真(True)	(not (True and True))为False

示例：

```
x = True
y = False
```

执行以下逻辑运算：

```
print("x and y =", x and y)          # 逻辑与
print("x or y =", x or y)            # 逻辑或
print("not x =", not x)              # 逻辑非
print("not y =", not y)              # 逻辑非
```

以上实例的执行结果为：

```
x and y = False
x or y = True
not x = False
not y = True
```

需要注意以下两点。

(1) x > 1 and x < 5 是判断某数x是否大于1且小于5的逻辑表达式。

(2) 如果逻辑表达式的操作数不是逻辑值True和False，Python 将非0视为真，0视为假进行运算。

例如，当 a = 0 和 b = 4 时，a and b 结果为假(0)，a or b结果为真：

```
>>>a = 0
>>>b = 4
>>>print(a and b)                   # 结果为0
0
>>>print(a or b)                    # 结果为4
4
```

说明：在Python中，or运算符是从左到右计算表达式，并返回第一个为真的值。

Python 中，逻辑值True作为数值时等于1，逻辑值False作为数值时等于0。

例如：

```
True + 5                    # 结果为6
6
```

由于True作为数值等于1，因此True＋5的结果为6。

```
False + 5                   # 结果为 5
5
```

逻辑值False作为数值等于0，因此False＋5的结果为5。

4. 赋值运算符

赋值运算符“=”的一般格式为：

```
变量 = 表达式
```

它表示将右侧表达式的计算结果赋值给左侧的变量。例如：

```
i = 3 * (4 + 5)             # i的值变为27
```

说明：

(1) 赋值运算符左侧必须是变量，右侧可以是常量、变量、函数调用，或由这些组成的表达式。例如：

```
x = 10
y = x + 10
y = func()
```

这些都是合法的赋值表达式。

(2) 赋值符号“=”与数学中的等号不同，它并不表示相等的含义。例如，x = x + 1是合法的(虽然在数学上不合法)，其含义是取出变量x的值加1，然后将结果存回变量x中。

赋值运算符的说明请参见表2-9。

表2-9　Python中的赋值运算符

运 算 符	说　　明	示　　例
=	直接赋值	c = a
＋=	加法赋值	c＋= a等同于c = c＋a
−	减法赋值	c − = a等同于c = c − a
*=	乘法赋值	c *= a等同于c = c * a
/=	除法赋值	c /= a等同于c = c / a
%=	取模赋值	c %= a等同于c = c % a

(续表)

运 算 符	说　　明	示　　例
**=	指数幂赋值	c **= a等同于c = c ** a
//=	整除赋值	c //= a等同于c = c // a

5. 位运算符

位(bit)是计算机中表示信息的最小单位，位运算符用于对位进行操作。在Python中，位运算符包括：按位与(&)、按位或(|)、按位异或(^)、按位求反(~)、左移(<<)和右移(>>)。位运算符对操作数的二进制形式逐位进行运算，参与位运算的操作数必须为整数。下面将逐一介绍这些运算符。假设有a = 60和b = 13，我们可以以二进制格式表示它们并进行运算：

```
a =       0011 1100
b =       0000 1101
a&b =     0000 1100
a | b =   0011 1101
a^b =     0011 0001
~a =      1100 0011
```

1) 按位与(&)

运算符“&”对其两边的操作数的对应位逐一进行逻辑与运算。每一位二进制数(包括符号位)都会参与运算。例如：

```
        a = 3
        b = 18
        c = a & b
        a  0000 0011
&       b  0001 0010
        c  0000 0010
```

所以，变量c的值为2。

2) 按位或(|)

运算符“|”对其两边的操作数的对应位逐一进行逻辑或运算。每一位二进制数(包括符号位)都会参与运算。例如：

```
        a = 3
        b = 18
        c = a | b
        a  0000 0011
|       b  0001 0010
        c  0001 0011
```

所以，变量c的值为19。

注意：尽管在位运算过程中是按位进行逻辑运算，但位运算表达式的结果并不是一个简单的逻辑值。

3) 按位异或(^)

运算符“^”对其两边的操作数的对应位逐一进行逻辑异或运算。每一位二进制数(包括符号位)都会参与运算。异或运算的定义是：若对应位不同，结果为1；若对应位相同，结果为0。例如：

```
        a = 3
        b = 18
        c = a ^ b
        a  0000 0011
^       b  0001 0010
        c  0001 0001
```

所以，变量c的值为17。

4) 按位求反(~)

运算符“~”是一元运算符，用于对操作数的对应位逐一取反。例如：

```
        a = 3
        c = ~a
^       a  00000011
        c  11111100
```

所以，变量c的值为 - 4。对于采用补码形式的带符号二进制数，最高位为1表示负数。

5) 左移(<<)

设a和n是整型变量，左移运算的一般格式为：a << n。其含义是将a的二进制位向左移动n位，移出的高n位被舍弃，而最低位补上n个0。

例如，当a = 7时，a的二进制形式为0000 0000 0000 0111。执行x = a << 3运算后，x的值为0000 0000 0011 1000，其十进制值为56。

左移一个二进制位，相当于将数值乘以2；左移n个二进制位，相当于将数值乘以2^n。

需要注意的是，左移运算可能会导致溢出问题。由于整数的最高位是符号位，当进行左移时，如果符号位不变，则相当于乘以2；但如果符号位发生变化，就会导致溢出。

6) 右移(>>)

设a和n是整型变量，右移运算的一般格式为：a >> n。其含义是将a的二进制位向右移动n位，移出的低n位被舍弃，而高n位则根据类型补充。若a是有符号整型，则高位补充符号位；如果a是无符号整型，则高位补充0。

右移一个二进制位相当于将数值除以2；右移n个二进制位则相当于将数值除以2^n。例如：

```
>>>a = 7
>>>x = a >> 1
>>>print(x)                    # 输出结果: 3
```

在上述例子中，a = 7进行x = a >> 1运算后，x的值为3。

6. 成员运算符

除了前面讨论的运算符，Python还提供了成员运算符，用于判断某个元素是否存在于序列中。成员运算符的详细信息如表2-10所示。

表2-10　Python中的成员运算符

运 算 符	说　　明	示　　例
x in y	如果x是序列y的成员，计算结果为True；否则为False	3 in [1, 2, 3, 4]计算结果为True 5 in [1, 2, 3, 4]计算结果为False
not in	如x不是序列y的成员，计算结果为True；否则为False	3 not in [1, 2, 3, 4]计算结果为False 5 not in [1, 2, 3, 4]计算结果为True

7. 标识运算符

标识运算符用于比较两个对象的内存位置。标识运算符的详细信息如表2-11所示。

表2-11　Python中的标识运算符

运 算 符	说　　明	示　　例
is	如果操作符两侧的变量指向相同的对象，计算结果为True；否则为False	如果id(x)的值等于id(y)，则x is y结果为True；否则为False
is not	如果操作符两侧的变量指向不同的对象，计算结果为True；否则为False	如果id(x)不等于id(y)，则x is not y结果为True；否则为False

8. 运算符的优先级

在一个表达式中，如果包含多种运算，将按照预先确定的顺序对各个部分进行计算和解析，这个顺序被称为运算符的优先级。当表达式中存在不止一种运算符时，将根据表2-12中的优先级规则进行计算(该表列出了所有运算符，从最高优先级到最低优先级)。

表2-12　Python运算符的优先级

优 先 级	运 算 符	说　　明
1	**	幂
2	~、+、-	求反、一元加号和减号
3	*、/、%、//	乘、除、取模和整除
4	+、-	加法和减法
5	<<、>>	左、右按位移
6	&	按位与

(续表)

优 先 级	运 算 符	说　　明
7	^、|	按位异或和按位或
8	<=、 >、 >=	比较(关系)运算符
9	<>、 ==、!=	比较(关系)运算符
10	=、 +=、 -=、 *=、 /=、 %=、 //=、 **=	赋值运算符
11	is、 is not	标识运算符
12	in、not in	成员运算符
13	not、or、and	逻辑运算符

2.3.2　表达式

Python语言中的表达式与其他编程语言的表达式没有显著区别。每个符合Python语言规则的表达式都会计算出一个确定的值。常量、变量的运算以及函数的调用都可以构成表达式。在本书后续章节中，我们将介绍的序列、函数和对象也可以成为表达式的一部分。

2.4　序列数据结构

在序列中，每个元素都被赋予了一个特定的数字，用以标识其在序列中的位置，这个数字被称为元素的“索引”或“位置”。第一个索引为0，第二个索引为1，以此类推。序列支持的操作包括索引、切片、加法、乘法以及成员检查。此外，Python 内置了确定序列长度以及查找最大和最小元素的方法。接下来将介绍 Python 中的列表、元组、字典和集合。

2.4.1　列表

列表(list)是Python中使用最广泛的数据类型。列表中的数据项不需要具有相同的类型。虽然列表类似于其他语言中的数组，但其功能要强大得多。

创建一个列表，只需将逗号分隔的不同数据项用方括号括起来即可。例如：

```
list1 = ['泰山', '嵩山', 1997, 2000]
list2 = [1, 2, 3, 4, 5]
list3 = ["a", "b", "c", "d"]
```

列表的索引从0开始，可以进行截取(切片)、组合等操作。

1. 访问列表中的值

使用下标索引可以访问列表中的值，也可以使用方括号进行切片。下面的示例演示了如何访问列表中的元素：

```
list1 = ['泰山', '嵩山', 1997, 2025]
list2 = [1, 2, 3, 4, 5, 6, 7]
print("list1[0]:", list1[0])
print("list2[1:5]:", list2[1:5])
```

输出结果：

```
list1[0]: 泰山
list2[1:5]: [2, 3, 4, 5]
```

2. 更新列表

可以对列表中的数据项进行修改或更新。例如：

```
list1 = ['泰山', 'mountain', 1997, 2025]
print("Value available at index 2:")
print(list1[2])
list1[2] = 2026
print("New value available at index 2:")
print(list1[2])
```

输出结果：

```
Value available at index 2:
1997
New value available at index 2:
2026
```

3. 删除列表中的元素

可以通过三种方法来删除列表中的元素。

- 方法1：使用del语句删除列表中的元素。示例如下：

```
list1 = ['泰山', '嵩山', 1997, 2025]
print(list1)
del list1[2]
print("After deleting value at index 2:")
print(list1)
```

输出结果：

```
['泰山', '嵩山', 1997, 2025]
After deleting value at index 2:
['泰山', '嵩山', 2025]
```

- 方法2：使用remove()方法删除列表中的元素。示例如下：

```
list1 = ['泰山', '嵩山', 1997, 2025]
list1.remove(2025)
list1.remove('嵩山')
```

```
print(list1)
```

输出结果：

```
['泰山', 1997]
```

- 方法3：使用 pop()方法删除列表的指定位置的元素(无参数时删除最后一个元素)。示例如下：

```
list1 = ['泰山', '嵩山', 1997, 2025]
list1.pop(2)                    # 删除位置2的元素1997
list1.pop()                     # 删除最后一个元素2025
print(list1)
```

输出结果：

```
['泰山', '嵩山']
```

4. 添加列表元素

可以使用append()方法在列表末尾添加元素。示例如下：

```
list1 = ['泰山', '嵩山', 1997, 2025]
list1.append(2026)
print(list1)
```

输出结果：

```
['泰山', '嵩山', 1997, 2025, 2026]
```

5. 定义多维列表

多维列表可以视为列表的嵌套结构，即多维列表的元素也是列表，但其维度比父列表低一层。对于二维列表(类似于其他编程语言中的二维数组)，其元素值是一维列表；而三维列表的元素值则是二维列表。例如，定义一个二维列表如下：

```
list2 = [["泰山", "嵩山"], ["长江", "黄河"]]
```

二维列表相较于一维列表多了一个索引，可以通过以下方式获取元素：

```
列表名[索引1][索引2]
```

例如，定义一个3行18列的二维列表，并打印出元素值：

```
rows = 3
cols = 18
matrix = [[0 for col in range(cols)] for row in range(rows)]      # 列表生成式

for i in range(rows):
    for j in range(cols):
        matrix[i][j] = i * 3 + j
```

```
        print(matrix[i][j], end=' ')
    print('\n')
```

输出结果：

```
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
```

列表生成式是Python内置的一种功能强大的表达式，用于生成列表。如果要生成列表[1, 4, 9, 16, 25, 36, 49, 64, 81]，可以使用列表生成式将要生成的元素 x * x 放在前面，后面跟上 for 循环，如下所示：

```
list1 = [x * x for x in range(1, 10)]
```

这样就可以创建出所需的列表。

6. 列表的操作符

列表的＋和*操作符与字符串类似。＋用于组合列表，而*用于重复列表。Python列表的操作符如表2-13所示。

表2-13　Python列表的操作符

表 达 式	描　　述	结　　果
len([1, 2, 3])	长度	3
[1, 2, 3]＋[4, 5, 6]	组合	[1, 2, 3, 4, 5, 6]
['Hi!'] * 4	重复	['Hi!', 'Hi!', 'Hi!', 'Hi!']
3 in [1, 2, 3]	元素是否存在于列表中	True
for x in [1, 2, 3]:print(x, end=" ")	迭代	1 2 3

Python列表的内置函数和方法如表2-14所示(假设列表名为list)。

表2-14　Python列表的方法和内置函数

方法和函数	功　　能
list.append(obj)	在列表末尾添加新的对象
list.count(obj)	统计某个元素在列表中出现的次数
list.extend(seq)	在列表末尾一次性追加另一个可迭代对象中的多个值(扩展原列表)
list.index(obj)	从列表中查找某个值第一个匹配项的索引位置
list.insert(index, obj)	在指定索引位置插入对象
list.pop(index)	移除列表中的一个元素(默认是最后一个元素)，并返回该元素的值

(续表)

方法和函数	功　　能
list.remove(obj)	移除列表中某个值的第一个匹配项
list.reverse()	反转列表中元素的顺序
list.sort([func])	对原列表进行排序
len(list)	内置函数，返回列表元素的个数
max(list)	内置函数，返回列表元素的最大值
min(list)	内置函数，返回列表元素的最小值
list(seq)	内置函数，将可迭代对象转换为列表

2.4.2　元组

Python的元组(tuple)与列表类似，但有一个重要的区别：元组的元素不可修改。元组使用小括号()来定义，而列表则使用方括号[ ]。元组中的元素类型可以不同。

1. 创建元组

创建元组非常简单，只需在小括号中添加元素，并用逗号隔开即可。示例如下：

```
tuple1 = ('泰山', '嵩山', 1997, 2025)
tuple2 = (1, 2, 3, 4, 5)
tuple3 = ('a', 'b', 'c', 'd')
```

如果要创建一个空元组，只需使用一对空括号：

```
tuple1 = ()
```

当元组中只包含一个元素时，需要在该元素后添加逗号，以区分它与普通的括号表达式。例如：

```
tuple1 = (50,)
```

元组与字符串类似，下标索引从0开始，可以进行切片、组合等操作。

2. 访问元组

可以使用下标索引访问元组中的值。示例如下：

```
tup1 = ('泰山', '嵩山', 1997, 2025)
tup2 = (1, 2, 3, 4, 5, 6, 7)

print("tup1[0]:", tup1[0])          # 输出元组的第一个元素
print("tup2[1:5]:", tup2[1:5])      # 切片，输出第二个元素到第五个元素
print(tup2[2:])                     # 切片，输出从第三个元素开始的所有元素
```

```
print(tup2 * 2)                    # 输出元组重复两次
```

输出结果：

```
tup1[0]: 泰山
tup2[1:5]: (2, 3, 4, 5)
(3, 4, 5, 6, 7)
(1, 2, 3, 4, 5, 6, 7, 1, 2, 3, 4, 5, 6, 7)
```

3. 元组连接

虽然元组中的元素值不可修改，但可以对元组进行连接组合。示例如下：

```
tup1 = (12, 34, 56)
tup2 = (78, 90)
# tup1[0] = 100              # 尝试修改元组元素的操作是非法的
tup3 = tup1 + tup2           # 连接元组，创建一个新的元组
print(tup3)
```

输出结果：

```
(12, 34, 56, 78, 90)
```

4. 删除元组

元组中的元素值不允许被删除，但可以使用del语句来删除整个元组。示例如下：

```
tup = ('泰山', '嵩山', 1997, 2025)
print(tup)
del tup                      # 删除整个元组
print("After deleting tup:")
print(tup)
```

在以上示例中，元组被删除后，输出会产生异常信息，结果如下：

```
('泰山', '嵩山', 1997, 2025)
After deleting tup:
NameError: name 'tup' is not defined
```

5. 元组运算符

与字符串类似，元组之间可以使用＋和*运算符进行运算。这意味着元组可以被组合和复制，运算后会生成一个新的元组。表2-15所示为Python元组的运算符总结。

表2-15　Python元组的运算符

表 达 式	描　　述	结　　果
len((1, 2, 3))	计算元素个数	3
(1, 2, 3) + (4, 5, 6)	连接元组	(1, 2, 3, 4, 5, 6)

(续表)

表 达 式	描　　述	结　　果
('a', 'b') * 4	复制元组	('a', 'b', 'a', 'b', 'a', 'b')
3 in (1, 2, 3)	检查元素是否存在	True
for x in (1, 2, 3): print(x, end=' ')	遍历元组	1 2 3

Python元组包含了表2-16所示的内置函数。

表2-16　Python元组的内置函数

函 数	描　　述	函 数	描　　述
len(tuple)	计算元组中的元素个数	max(tuple)	返回元组中元素的最大值
min(tuple)	返回元组中元素的最小值	tuple(seq)	将可迭代对象转换为元组

示例如下：

```
tup1 = (12, 34, 56, 6, 77)
y = min(tup1)
print(y)
```

输出结果：

```
6
```

可以使用元组一次性对多个变量进行赋值。例如：

```
(x, y, z) = (1, 2, 3)          # 或者使用x, y, z = 1, 2, 3也可以
print(x, y, z)
```

输出结果：

```
1 2 3
```

如果想要实现x和y的交换，可以使用以下方法：

```
x = 34
y = 21
print(x, y)                    # 输出结果为: 34 21

# 交换 x 和 y
x, y = y, x
print(x, y)                    # 输出结果为: 21 34
```

6. 元组与列表转换

由于元组的元素不可改变，可以将元组转换为列表，以便对数据进行修改。实际上，

列表、元组和字符串之间可以互相转换，使用的函数包括str()、tuple()和list()。

可以使用以下方法将元组转换为列表：

列表对象＝list(元组对象)

```
tup = (1, 2, 3, 4, 5)
list1 = list(tup)                # 元组转为列表
print(list1)                     # 输出结果为: [1, 2, 3, 4, 5]
```

可以使用以下方法将列表转换为元组：

元组对象＝tuple(列表对象)

```
nums = [1, 3, 5, 7, 8, 13, 20]
print(tuple(nums))          # 返回: (1, 3, 5, 7, 8, 13, 20)
```

将列表转换为字符串的方法如下：

```
nums = [1, 3, 5, 7, 8, 13, 20]
str1 = str(nums)            # 列表转换为字符串，返回含中括号及逗号的' [1, 3, 5, 7, 8, 13, 20] '字符串
print(str1[2])              # 返回字符串中的第三个字符，即逗号
```

使用连接符可以将列表元素转换为字符串。例如，有一个包含山名的列表，可以将其转换为连接字符串：

```
num2 = ['泰山', '嵩山', '华山', '衡山']
str2 = "8".join(num2)       # 用'8'连接元素
print(str2)                 # 输出结果为: '泰山8嵩山8华山8衡山'

str2 = "".join(num2)        # 用空字符连接元素
print(str2)                 # 输出结果为:'泰山嵩山华山衡山'
```

2.4.3　字典

Python字典(dict)是一种可变容器模型，能够存储任意类型的对象，如字符串、数字、元组等其他容器模型。字典也被称为关联数组或哈希表。

1. 创建字典

字典由键及其对应的值(key => value)组成。键与值之间用冒号分隔，键/值对之间用逗号分隔，整个字典用大括号包围。基本语法如下：

```
d = {key1: value1, key2: value2}
```

这里需要注意的是，键必须是唯一的，但值可以重复。值可以是任何数据类型，而键必须是不可变的，例如字符串、数字或元组。

以下是一个简单的字典实例：

```
dict1 = {'xmj': 40, 'zhang': 91, 'wang': 80}
```

也可以使用以下方式创建字典：

```
dict2 = {'abc': 456}
dict3 = {'abc': 123, 98.6: 37}
```

字典具有以下特性。

- 值的类型：字典的值可以是任何Python对象，例如字符串、数字、元组等。
- 键的唯一性：不允许同一个键出现两次。如果在创建字典时同一个键被赋值两次，后一个值将覆盖前面的值。示例如下：

```
dict1 = {'Name': 'xmj', 'Age': 17, 'Name': 'Manni'}
print("dict['Name']: ", dict1['Name'])
```

以上实例的输出结果为：

```
dict['Name']: Manni
```

- 键的不可变性：字典的键必须是不可变的类型，因此可以使用数字、字符串或元组作为键，但不能使用列表。示例如下：

```
dict2 = {[ 'Name' ]: 'Zara', 'Age': 7}
```

该实例将导致错误，输出如下：

```
Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    dict2 = {[ 'Name' ]: 'Zara', 'Age': 7}
TypeError: unhashable type: 'list'
```

2. 访问字典中的值

要访问字典中的值，可以将相应的键放入方括号中。示例如下：

```
dict1 = {'Name': '王燕', 'Age': 19, 'Class': '计算机二班'}
print("dict['Name']: ", dict1['Name'])
print("dict['Age']: ", dict1['Age'])
```

输出结果：

```
dict['Name']:  王燕
dict['Age']:  19
```

如果尝试使用字典中不存在的键来访问数据，将会出现错误信息。示例如下：

```
dict1 = {'Name': '王燕', 'Age': 19, 'Class': '计算机一班'}
print("dict['sex']: ", dict1['sex'])
```

由于字典中没有sex这个键，以上示例将输出错误信息：

```
Traceback (most recent call last):
    File "<pyshell#0>", line 2, in <module>
        print("dict['sex']: ", dict1['sex'])
KeyError: 'sex'
```

3. 修改字典

向字典中添加新内容的方法包括增加新的键/值对，或修改、删除已有的键/值对。示例如下：

```
dict1 = {'Name': '王燕', 'Age': 19, 'Class': '计算机一班'}
dict1['Age'] = 20                    # 更新已有的键/值对
dict1['School'] = '南京理工大学'      # 增加新的键/值对
print("dict['Age']: ", dict1['Age'])
print("dict['School']: ", dict1['School']);
```

输出结果：

```
dict['Age']:  20
dict['School']:  南京理工大学
```

4. 删除字典元素

del()方法允许通过键从字典中删除元素(条目)，而clear()方法可以清空字典中的所有元素。示例如下：

```
dict1 = {'Name': '王燕', 'Age': 19, 'Class': '计算机一班'}
del dict1['Name']                # 删除键为'Name'的元素
dict1.clear()                    # 清空字典中的所有元素
del dict1                        # 删除整个字典
```

5. in 运算

在字典中，in 运算符用于判断某个键是否存在于字典中，不能用于判断值是否存在。其功能类似于has_key(key)方法。示例如下：

```
dict1 = {'Name': '王燕', 'Age': 19, 'Class': '计算机一班'}
print('Age' in dict1)            # 等效于print(dict1.has_key('Age'))
```

输出结果：

```
True
```

6. 获取字典中的所有值

values() 方法返回字典中所有值的列表。示例如下：

```
dict1 = {'Name': '王燕', 'Age': 19, 'Class': '计算机一班'}
print(dict1.values())
```

输出结果：

```
dict_values(['王燕', 19, '计算机一班'])
```

7. items() 方法

items()方法将字典中的每对键(key)和值(value)组合成一个元组，并将这些元组放在一个列表中返回。示例如下：

```
dict1 = { 'Name': '王燕', 'Age': 19, 'Class': '计算机一班'}
for key, value in dict1.items():
    print(key, value)
```

输出结果：

```
Name 王燕
Age 19
Class 计算机一班
```

需要注意的是，字典打印出的顺序可能与创建时的顺序不同，这并不是错误。字典中的元素是无序的(因为不需要通过位置查找元素)，因此在存储时进行了优化，以提高字典的存储和查询效率。这也是字典与列表的一个重要区别：列表保持元素的相对顺序，即序列关系，而字典则是完全无序的，也称为非序列。如果希望保持集合中元素的顺序，应使用列表而非字典。

字典的内置函数和方法如表2-17所示(假设字典名为 dict1)。

表2-17　字典的内置函数和方法

函数和方法	描　　述
dict1.clear()	删除字典内的所有元素
dict1.copy()	返回一个字典的副本(浅复制)
dict1.fromkeys(seq, value)	创建一个新字典，以序列seq中的元素作为字典的键，所有键对应的初始值为value
dict1.get(key, default=None)	返回指定键的值，如果键不存在，则返回default值
dict1.has_key(key)	如果键在字典dict1中，返回True，否则返回False(注意：此方法在Python3.0以后已被删除)
dict1.items()	以列表返回可遍历的(键，值)字典视图对象
dict1.keys()	以列表返回字典中的所有键
dict1.setdefault(key, default=None)	类似于get()方法，但如果键不存在于字典中，则添加该键并将值设为default
dict1.update(dict2)	将字典dict2的键/值对更新到dict1中
dict1.values()	以列表返回字典中的所有值
cmp(dict1, dict2)	内置函数，比较两个字典的元素(注意：在Python 3中已被移除)
len(dict)	内置函数，计算字典中元素的个数，即键的总数
str(dict)	内置函数，输出字典的可打印字符串表示
type(variable)	内置函数，返回输入变量的类型，如果变量是字典，则返回字典类型

2.4.4　集合

集合(set)是一个无序且不重复元素的序列。集合的基本功能包括进行成员测试和删除重复元素。

1. 创建集合

可以使用大括号“{}”或者set()函数来创建集合。需要注意的是，创建一个空集合时必须使用set()，而不能使用{ }，因为{ }是用来创建一个空字典。

例如，可以这样创建一个集合：

```
student = {'泰山', '嵩山', '华山', '衡山', '泰山', '昆仑山'}
print(student)                    # 输出集合，重复的元素会被自动去掉
```

输出结果：

```
{'衡山', '华山', '昆仑山', '泰山', '嵩山'}
```

2. 成员测试

可以使用in关键字进行成员测试，判断某个元素是否在集合中。例如：

```
if '紫金山' in student:
    print('紫金山在集合中')
else:
    print('紫金山不在集合中')
```

输出结果：

```
紫金山不在集合中
```

3. 集合运算

可以使用运算符“-”、“|”和“&”来进行集合的差集、并集和交集运算。示例如下：

```
# set 可以进行集合运算
a = set('abcd')
b = set('cdef')
print(a)
print("a 和 b 的差集：", a-b)                # a和b的差集
print("a 和 b 的并集：", a | b)              # a和b的并集
print("a 和 b 的交集：", a & b)              # a和b的交集
print("a 和 b 中不同时存在的元素：", a ^ b)   # a和b中不同时存在的元素
```

输出结果：

```
{'c', 'b', 'a', 'd'}
a 和 b 的差集：  {'b', 'a'}
a 和 b 的并集：  {'a', 'e', 'c', 'b', 'd', 'f'}
a 和 b 的交集：  {'c', 'd'}
a 和 b 中不同时存在的元素：  {'a', 'e', 'b', 'f'}
```

2.5 课后实践

本章全面介绍了Python的基础知识，涵盖了数据类型、变量和常量、运算符与表达式，以及序列数据结构。从基本的数值、字符串和布尔类型，到复杂的列表、元组、字典和集合，本章详细讲解了各种数据类型的特性和用法。同时，深入探讨了变量与常量的概念，以及各种运算符的使用方法。这些基础内容为用户提供了系统的Python语法知识，为今后的编程实践奠定了坚实的基础。掌握这些核心概念，将有助于用户更好地理解和应用Python编程。

在下面的课后实践部分，用户可以通过动手实现几个应用实例并完成课后的思考练习，巩固所学的知识。

一、应用实例

【例2-1】生成不重复随机数。

分析：下面使用三种不同方法生成指定数量的不重复随机数，并对它们进行性能比较。第一种方法使用列表和计数器，第二种方法使用列表和长度检查，第三种方法使用集合。代码包含一个性能测试函数，用于测量每种方法在生成10 000次随机数集合时的执行时间。通过比较这些执行时间，可以评估哪种方法在生成不重复随机数时更加高效，从而为类似任务选择最优的实现方式。

```
import random
import time

def generate_random_numbers_v1(number, start, end):
    """
    使用列表和计数器来生成指定数量的介于给定范围内的不重复随机数
    """
    data = [ ]
    count = 0
    while count < number:
        element = random.randint(start, end)
        if element not in data:
            data.append(element)
            count += 1
    return data

def generate_random_numbers_v2(number, start, end):
    """
    使用列表和长度检查来生成指定数量的介于给定范围内的不重复随机数
    """
    data = [ ]
    while len(data) < number:
```

```
            element = random.randint(start, end)
            if element not in data:
                data.append(element)
        return data

def generate_random_numbers_v3(number, start, end):
    """
    使用集合来生成指定数量的介于给定范围内的不重复随机数
    """
    data = set( )
    while len(data) < number:
        data.add(random.randint(start, end))
    return data

# 性能测试
def performance_test(func, iterations=10000):
    start_time = time.time()
    for _ in range(iterations):
        func(50, 1, 100)
    elapsed_time = time.time() - start_time
    print(f"{func.__name__} 执行时间: {elapsed_time:.4f} 秒")

# 运行性能测试
performance_test(generate_random_numbers_v1)
performance_test(generate_random_numbers_v2)
performance_test(generate_random_numbers_v3)
```

【例2-2】使用字典统计歌曲中出现的高频词。

分析：我们可以创建一个空字典，然后遍历数据中的单词，以单词作为键，计数器作为对应的值。在遍历每个单词时，需要判断该单词是否已经在字典中。如果存在，则将计数器加1；如果不存在，则创建这个键，并将计数器初始化为1。可以将这个核心功能封装成一个函数，只需将数据传入该函数，它就会返回一个字典。

(1) 使用Windows系统自带的记事本工具创建KMSWHS.txt文件，用于保存歌词。

```
I heard he sang a good song I heard he had a style
And so I came to see him to listen for a while
And there he was this young boy a stranger to my eyes
Strumming my pain with his fingers
Singing my life with his words
Killing me softly with his song
Killing me softly with his song
……(参见素材文件)
```

(2) 编写实例代码：

```
contents = open('KMSWHS.txt', 'r').read( )          # 读取全部歌词
cList = contents.split()

def histogram(contents):
    d = {}                                          # 创建一个空字典
    for w in contents:
        if w in d:
            d[w] += 1                               # 如果单词已存在，则计数加1
        else:
            d[w] = 1                                # 如果单词不存在，则创建键，计数器记为1
    return d

h = histogram(cList)
print(h)                                            # 打印单词频率字典
```

以上代码中open()函数用于打开文本文件，而read()方法则一次性读取文件内的所有数据。运行结果将输出每个单词及其出现的次数，如下所示：

```
{'I': 6, 'heard': 2, 'he': 5, 'sang': 1, 'a': 4, 'good': 1, 'song': 15, 'had': 1, 'style': 1, 'And': 2, 'so': 1, 'came': 1, 'to': 3, 'see': 1, 'him': 1, 'listen': 1, 'for': 1, 'while': 1, 'there': 1, 'was': 1, 'this': 1, 'young': 1, 'boy': 1, 'stranger': 1, 'my': 16, 'eyes': 1, 'Strumming': 4, 'pain': 4, 'with': 27, 'his': 26, 'fingers': 4, 'Singing': 4, 'life': 9, 'words': 8, 'Killing': 14, 'me': 14, 'softly': 14, 'Telling': 4, 'whole': 4, 'felt': 2, 'all': 1, 'flushed': 1, 'fever': 1, 'embarassed': 1, 'by': 1, 'the': 1, 'crowd': 1, "he'd": 1, 'found': 1, 'letters': 1, 'and': 1, 'read': 1, 'each': 1, 'one': 1, 'out': 1, 'loud': 1, 'prayed': 1, 'that': 1, 'would': 1, 'finish': 1, 'but': 1, 'just': 1, 'kept': 1, 'right': 1, 'on': 1, 'You': 1, 'will': 2, 'following': 1, 'hand': 1, 'yeeh': 1, 'saving': 1}
```

接下来，可以定义一个show函数，以便将结果以更友好的格式展示出来：

```
def show(wordsDict):
    dictList = [ ]
    for key, val in wordsDict.items():
        dictList.append((val, key))                 # 将每个键值对添加到列表中
    dictList.sort(reverse=True)                     # 倒序排序
    # 打印表头
    print('%-10s%10s' % ('word', 'count'))
    print('-' * 25)
    # 打印每个单词及其计数，条件是计数大于10
    for val, key in dictList:
        if val > 10:
            print('%-12s %3d' % (key, val))
# 调用 show 函数展示结果
show(h)
```

输出结果：

```
word                count
-------------------------
with                27
his                 26
my                  16
song                15
softly              14
me                  14
Killing             14
```

【例2-3】编写程序，用户输入6个数字和5个数字形成两个列表，合并后在列表末尾添加99和100，最后对列表进行降序排序并输出。

分析：用户通过键盘分别输入6个数字和5个数字，形成两个列表list1和list2。接着，将list2合并到list1中，并在list1的末尾添加两个数字99和100。最后，对list1进行降序排序，并输出最终的列表。

```
list1 = [ ]                          # 初始化一个空列表
list2 = [ ]                          # 初始化一个空列表
print("列表 list1：")
for i in range(6):
    # 循环6次，输入6个数字放到列表 list1 中
    x = int(input("请输入第" + str(i + 1) + "个元素："))          # 转换为整数
    list1.append(x)
print("列表 list2：")
for i in range(5):
    # 循环5次，输入5个数字放到列表 list2 中
    x = int(input("请输入第" + str(i + 1) + "个元素："))          # 转换为整数
    list2.append(x)
print("list1: ", list1)
print("list2: ", list2)
list1.extend(list2)                                              # 将列表 list2 合并到 list1 中
print("列表 list2 合并到 list1 中后的数据：", list1)
list1 += [99, 100]                                               # 在 list1 末尾添加99和100
print("加上99, 100后的 list1 的数据：", list1)
list1.sort(reverse=True)                                         # list1 降序排列
print("降序排列后最终列表 list1 中的数据：", list1)
```

输出结果：

```
列表 list1：
请输入第1个元素：3
请输入第2个元素：2
请输入第3个元素：5
```

```
请输入第4个元素：4
请输入第5个元素：3
请输入第6个元素：6
列表 list2：
请输入第1个元素：11
请输入第2个元素：23
请输入第3个元素：3
请输入第4个元素：4
请输入第5个元素：65
list1: [3, 2, 5, 4, 3, 6]
list2: [11, 23, 3, 4, 65]
列表 list2 合并到 list1 中后的数据： [3, 2, 5, 4, 3, 6, 11, 23, 3, 4, 65]
加上99, 100后的 list1 的数据： [3, 2, 5, 4, 3, 6, 11, 23, 3, 4, 65, 99, 100]
降序排列后最终列表 list1 中的数据： [100, 99, 65, 23, 11, 6, 5, 4, 4, 3, 3, 3, 2]
```

二、思考练习

1. Python 数据类型有哪些？它们各自的用途是什么？

2. 计算下列表达式的值(可在上机时验证)，设a=7，b=－2，c=4。

(1) 3 * 4 ** 5/2

(2) a * 3 % 2

(3) a % 3 + b * b－c//5

(4) b ** 2－4 * a * c

3. 求列表s = [9, 7, 8, 3, 2, 1, 55, 6]中的元素个数、最大值和最小值。如何在列表s中添加一个元素10？如何从列表s中删除元素55？

4. 元组与列表的主要区别是什么？对于元组s = (9, 7, 8, 3, 2, 1, 55, 6)，能否添加元素？

5. “二分法”是编程中常用的算法之一。例如，在猜数字游戏中，猜的数字是0～100的整数。首先，我们取0～100的中间数，即50，询问这个数是比要猜的数大还是小。如果小，则下一次取50～100的中间数；如果大，则取0～50的中间数。这个过程将持续进行，直到猜中正确的数字。请编写程序实现二分法猜数字的功能。